

Queueing Theory for Agentic LLM Serving

Lecture Notes for a Four-Hour Course

Serving Queue Theory Project

September 2026

Contents

About These Notes	3
1 Probabilistic Foundations: Arrivals, Service, and Little's Law	4
1.1 Queueing systems and performance measures	4
1.2 From a serving system to a queueing model	5
1.3 The exponential distribution and memorylessness	7
1.4 The Poisson process	8
1.5 PASTA: Poisson arrivals see time averages	9
1.6 The renewal-reward theorem	11
1.7 Little's law	12
1.8 Exercises	13
2 Markovian Queues: M/M/1, Processor Sharing, and Insensitivity	14
2.1 Continuous-time Markov chains	14
2.2 Birth–death processes and M/M/1	15
2.3 Processor sharing and state-dependent capacity	16
2.4 Insensitivity	18
2.5 The price at the decode stage	20
2.6 Exercises	20
3 The M/G/1 Queue and the Pollaczek–Khinchine Formula: the Price of a Miss	22
3.1 The M/G/1 queue and the residual service time	22
3.2 The Pollaczek–Khinchine formula	24
3.3 Mixtures of hits and misses	25
3.4 The price of a miss	26
3.5 Exercises	28
4 Closed Systems and Memory: Finite Populations, MVA, Campbell's Theorem, Eviction	29
4.1 Closed systems and mean value analysis	29
4.2 M/G/ ∞ and Campbell's theorem	32
4.3 Eviction: the covering knapsack problem	33
4.4 Exercises	35
5 Putting It Together: The Theory of the Paper, Step by Step	36
5.1 The model in one page	36
5.2 Where the variance of prefill work comes from	37
5.3 The decision problem and one price	37
5.4 The scheduler, step by step	40

5.5	What is exact and what is approximate	42
5.6	Exercises	43

About These Notes

Goal. These notes accompany a four-hour course for readers who know real analysis and mathematical statistics but have not met queueing theory. The goal is narrow: to be able to prove, unaided, the four results our research uses to model and analyse agentic LLM serving with queues. Each lecture takes about an hour, and every concept is defined before it is used. Measure-theoretic probability is not needed; we use random variables, expectation, conditional expectation, the strong law of large numbers (SLLN) and power series freely.

Background: why LLM serving is a queue. An LLM request is served in two phases. *Prefill* reads the whole prompt in one pass and produces the first token; the time from the request's arrival to its first token is the *time to first token* (TTFT). *Decode* then produces one token per step. Both phases keep the attention state of every prompt token, the *KV cache*, in device memory. When a later request starts with the same tokens, the stored state can be reused (a *hit*); if it has been evicted from memory, it must be recomputed (a *miss*). An agent program is a *session* that loops through *prefill* $\rightarrow$ *decode* $\rightarrow$ tool call $\rightarrow$ *prefill* $\rightarrow \dots$, and each request (a *turn*) is the previous prompt with a tool result appended. Requests arrive at random, wait for a server (the accelerator), receive service and leave. This is exactly the object of queueing theory.

Map. The four results of our research, and what these notes provide to prove them:

Research result	Needed from these notes
The decode stage prices only work	Little's law (Theorem 1.19), birth–death processes (Theorem 2.6), processor sharing and insensitivity (Theorem 2.14), Theorem 2.12, Proposition 2.16
The price of a miss	PASTA (Theorem 1.16), residual service time (Lemma 3.3), the Pollaczek–Khinchine formula (Theorem 3.5), Theorem 3.11
Finite population	closed systems, the arrival theorem (Theorem 4.3), mean value analysis (Theorem 4.5), Proposition 4.6
Price-blind eviction keys	the covering knapsack, the threshold rule (Theorem 4.16), Proposition 4.17; Campbell's theorem (Theorem 4.13) for the memory estimate

Plan. Lecture 1 covers arrival processes and Little's law; Lecture 2 Markovian queues and processor sharing; Lecture 3 the M/G/1 queue, the Pollaczek–Khinchine formula and the price of a miss; Lecture 4 closed systems and memory; and Lecture 5 assembles these tools into the theory of the paper, step by step. The exercises at the end of each lecture are meant to be solved before the next one. Good introductory textbooks are [12, 5]; [1] is the rigorous reference.

Lecture 1

Probabilistic Foundations: Arrivals, Service, and Little's Law

This lecture defines the vocabulary of queueing theory and proves three tools that every later lecture relies on: the memorylessness of the exponential distribution, the Poisson process together with PASTA, and Little's law. We assume undergraduate probability (conditional expectation, the strong law of large numbers, probability generating functions).

1.1 Queueing systems and performance measures

Definition 1.1 (Queueing system)

A **queueing system** consists of four elements.

1. **Customers** (or jobs): entities that arrive at the system, require some amount of processing, and leave. The arrival time of the k -th customer is a_k , numbered so that $0 \leq a_1 \leq a_2 \leq \dots$.
2. The **arrival process**: the sequence of arrival times $(a_k)_{k \geq 1}$, or equivalently the counting process $A(t) = \#\{k : a_k \leq t\}$.
3. A **server** and **service times**: the server is the resource that processes customers. Customer k needs time S_k when it has the server to itself. S_k is also called the **work** or service requirement of customer k .
4. A **waiting room** and a **service discipline**: the space where customers arriving at a busy server wait, and the rule that decides whom to serve, when, and how much.

Definition 1.2 (Service disciplines)

FIFO (first in, first out) serves one customer at a time, to completion, in order of arrival. **Processor sharing** (PS) serves all customers in the system simultaneously at equal rates; it is defined formally in Lecture 2. A discipline under which the server never idles while work is present is called **work-conserving**. Both FIFO and PS are work-conserving.

Definition 1.3 (Kendall notation)

A queueing system is written $A/S/c/K/N$. Here A is the distribution of the interarrival times, S the distribution of the service times, c the number of servers, K the system capacity (∞ if omitted), and N the size of the customer population (∞ if omitted). The distribution symbols are M (Markov: exponential and independent), D (deterministic: constant), and G (general: an arbitrary distribution, i.i.d.).

For example, $M/M/1$ has Poisson arrivals, exponential service and one server (Lecture 2); $M/G/1$ allows a general service distribution (Lecture 3); $M/M/1/N$ is a closed system with only N customers (Lecture 4); and $M/G/\infty$ is an infinite-server system with no waiting (Lecture 4).

Definition 1.4 (Performance measures)

Let d_k be the time at which customer k leaves the system.

1. The **sojourn time** (response time) is $W_k = d_k - a_k$.
2. The **waiting time** $W_{q,k}$ is the time from the arrival of customer k until its service first begins. For a FIFO single server, $W_k = W_{q,k} + S_k$.
3. The **number in system** is $N(t) = \#\{k : a_k \leq t < d_k\}$. The number in the waiting room is written $N_q(t)$.

Quantities in queueing theory are defined by two different kinds of averages, and the distinction will be essential several times.

Definition 1.5 (Time averages and customer averages)

The **time average** of the process $N(t)$ is $L = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t N(s) ds$, and the **customer average** of the sequence (W_k) is $W = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n W_k$ (when the limits exist). L_q and W_q are defined in the same way. If the limits are constants with probability one, then in the stationary regime they equal $\mathbb{E}[N]$ and $\mathbb{E}[W]$ respectively.

Definition 1.6 (Arrival rate, load, stability)

The **arrival rate** is $\lambda = \lim_{t \rightarrow \infty} A(t)/t$. The **load** (utilisation) of a single-server system with mean service time $\mathbb{E}[S]$ is

$$\rho = \lambda \mathbb{E}[S].$$

ρ is the amount of work arriving per unit time and is dimensionless. The system is **stable** if $N(t)$ has a proper limiting distribution (or at least $L < \infty$) and every customer leaves in finite time.

A single server processes at most one unit of work per unit time, so if $\rho > 1$ the unfinished work grows linearly and the system cannot be stable. For a work-conserving $G/G/1$ system (i.i.d. interarrival and service times) it is known that $\rho < 1$ is essentially necessary and sufficient for stability [15]. For the models in these notes we assume $\rho < 1$ as the stability condition and verify it model by model.

1.2 From a serving system to a queueing model

A queueing model of a serving system is a projection: it keeps what decides who waits and for how long, and drops the rest. Figure 1.1 shows the projection used throughout these notes. The stations it names (processor sharing, delay stations, closed loops) are defined formally in later

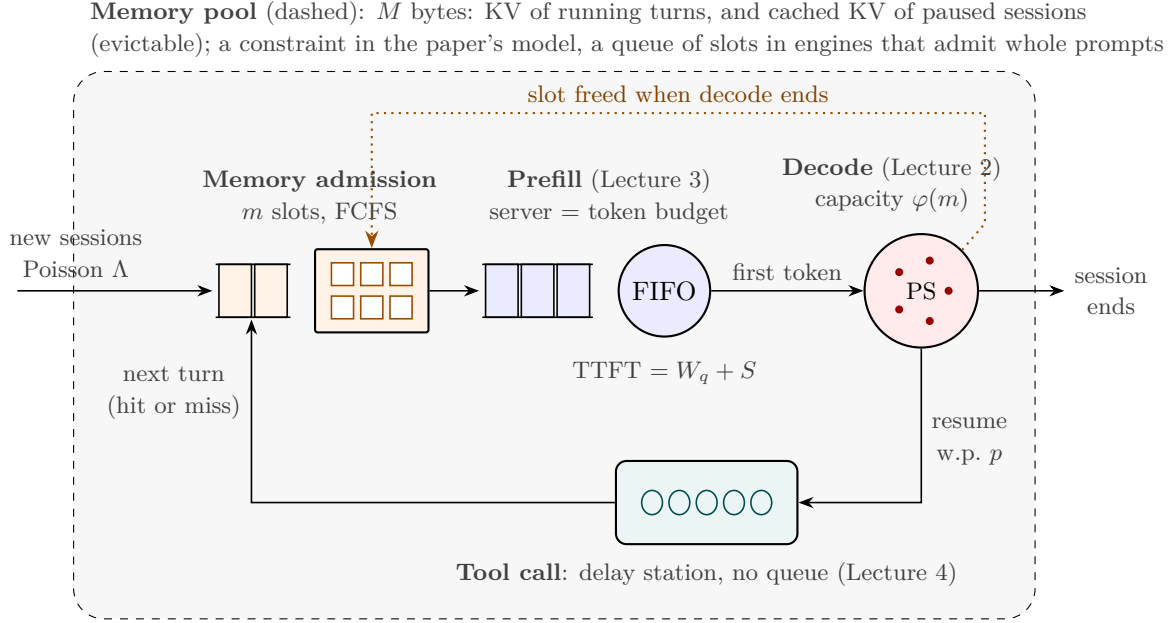


Figure 1.1: The serving system projected onto queues. Turns are the customers of the stations; sessions are the customers of the closed loop through the tool call. The dashed pool is shared memory: what it drops decides whether the next turn is a hit (short prefill work) or a miss (long prefill work). The memory-admission station (orange) is *not* in the paper's model: it appears in engines that admit a turn only when its whole context fits, in arrival order; a turn then holds a slot from the start of its prefill to the end of its decode (Section 5.5).

lectures; here they are described only informally, so that every later theorem has a place in the picture.

Customers at two levels. A *turn* is a customer of the prefill and decode stations: it arrives, waits, is served and leaves. A *session* is a customer at a coarser level: it circulates through prefill, decode and the tool call, and leaves only when the agent program ends. New sessions arrive from outside as a Poisson process of rate Λ , but the turns of a live session do not arrive independently of the state: the next turn exists only after the previous one has been served. The prefill and decode lectures look at turns; the lectures on closed systems, memory and admission look at sessions.

Three stations. *Prefill* is a single FIFO server whose capacity is the token budget of an engine step: serving engines by default give that budget to the oldest pending prefill first. With Poisson arrivals this is the M/G/1 queue of Lecture 3; with a finite number N of live sessions it is the closed system of Lecture 4. *Decode* is a processor-sharing station: every step advances every turn in the batch by one token, so the batch shares the device equally, with a total rate $\varphi(m)$ that depends on the batch size m (Lecture 2). The *tool call* is a delay station: every session calls its own tool at once, without queueing, for a random think time.

Memory is a constraint, and in some engines a queue. The pool of M bytes holds the KV cache of running turns and of paused sessions. In the paper's model nobody queues “at” the memory: the pool only bounds how much state can stay resident, and counting how much is resident on average is a calculation about sessions that stay in the system for a random time, which Lecture 4 does with the M/G/ ∞ model and Campbell's theorem. Many engines, however,

admit a waiting turn only when its whole context fits, and in arrival order; vLLM does so by default (its scheduler checks that the full input sequence fits before admitting a request, and stops at the first request that does not). Memory is then a station as well. If every turn needs about c bytes, the pool holds $m = \lfloor M/c \rfloor$ slots; a turn holds its slot from the start of its prefill to the end of its decode, so the station is a multi-server queue $\cdot/G/m$ whose service time is the holding time $S + D$, and a turn that does not fit blocks everyone behind it (head-of-line blocking, now on memory). Two caveats: with unequal contexts a turn needs several slots and the station has no closed form; and in such engines the cache of paused sessions does not take slots away. Cached blocks that no running turn uses count as free: an admission that needs them evicts the least recently used ones, so the slots compete with the cache by turning hits into misses, not by blocking admission. (The paper's model, which lets a controller keep or drop a paused state at a price, is a policy these engines do not implement: their KV policy is LRU on the free blocks.) On the paper's testbed with long contexts this station, not the prefill queue, set the waits (Section 5.5). The $M/M/m$ queue is an exercise of Lecture 2.

The control changes the service law. In classical queueing theory the service-time law is given. Here the KV policy chooses it: a turn whose prefix is still resident is a hit with short prefill work, and one whose prefix was dropped is a miss that recomputes the whole context. The policy therefore sets the distribution of S at the prefill station, and the central question of Lecture 3 is how a performance measure changes when that distribution changes.

Performance. The time to first token is the sojourn time at the prefill station, $\text{TTFT} = W_q + S$. The objective is the mean number of turns in the two stations, $L = L_P + L_D$; by Little's law (Theorem 1.19) it is the same as the mean delay per unit time.

What the projection drops. Work is measured in seconds rather than tokens and memory blocks; the budget left to prefill by the decode batch fluctuates, and the model replaces it by an average; the paper's model has no memory-admission station, so a turn whose KV does not fit waits for free blocks without the model seeing it; and session feedback makes turn arrivals non-Poisson. Lecture 5 returns to these gaps once the tools are in place.

1.3 The exponential distribution and memorylessness

Definition 1.7 (Exponential distribution)

A random variable $X \geq 0$ with $\mathbb{P}(X > t) = e^{-\mu t}$ for $t \geq 0$ has the **exponential distribution** with **rate** $\mu > 0$, written $X \sim \text{Exp}(\mu)$. Then $\mathbb{E}[X] = 1/\mu$ and $\mathbb{E}[X^2] = 2/\mu^2$.

Proposition 1.8 (Memorylessness [20])

If $X \sim \text{Exp}(\mu)$, then for all $s, t \geq 0$

$$\mathbb{P}(X > s + t \mid X > s) = \mathbb{P}(X > t). \quad (1.1)$$

Conversely, if a random variable with $0 < X < \infty$ a.s. satisfies (1.1), then $X \sim \text{Exp}(\mu)$ for some $\mu \in (0, \infty)$.

Proof. The forward direction is $e^{-\mu(s+t)}/e^{-\mu s} = e^{-\mu t}$. Converse: let $G(t) = \mathbb{P}(X > t)$. Then (1.1) reads $G(s + t) = G(s)G(t)$. G is nonincreasing and right-continuous, and $G(0) = \mathbb{P}(X > 0) = 1$.

By induction $G(nt) = G(t)^n$, hence $G(1) = G(1/n)^n$ and $G(m/n) = G(1)^{m/n}$. If $G(1) = 0$, then $G(1/n) = 0$ for every n , which contradicts $G(1/n) \rightarrow G(0) = 1$ by right-continuity. If $G(1) = 1$, then $G(r) = 1$ for every rational r , and by monotonicity $G \equiv 1$, that is $X = \infty$ a.s., contradicting the assumption. Hence $G(1) = e^{-\mu}$ for some $\mu \in (0, \infty)$, and $G(r) = e^{-\mu r}$ on the rationals. For arbitrary t , take rationals $r \downarrow t$; right-continuity gives $G(t) = e^{-\mu t}$. $\square$

Memorylessness says that the remaining service time of a customer who has already received s units of service has the same distribution as the service time of a new customer. This is why, in a system with exponential service, the number in system $N(t)$ alone determines the future, and $N(t)$ is a continuous-time Markov chain (Lecture 2). The following fact is also used often.

Lemma 1.9 (Minimum of exponentials [20])

If $X_i \sim \text{Exp}(\mu_i)$, $i = 1, \dots, n$, are independent, then $\min_i X_i \sim \text{Exp}(\sum_i \mu_i)$.

Proof. $\mathbb{P}(\min_i X_i > t) = \prod_i \mathbb{P}(X_i > t) = e^{-(\sum_i \mu_i)t}$. $\square$

1.4 The Poisson process

Definition 1.10 (Counting process and Poisson process)

A **counting process** $A = (A(t))_{t \geq 0}$ is a nondecreasing, right-continuous, $\mathbb{N}$ -valued process with $A(0) = 0$ and jumps of size one only. A counting process A is a **Poisson process** with **rate** $\lambda > 0$ if

1. (**independent increments**) for disjoint intervals $(s_1, t_1], \dots, (s_n, t_n]$ the increments $A(t_j) - A(s_j)$ are independent;
2. (**stationary Poisson increments**) for all $s \geq 0$, $t > 0$, $A(s+t) - A(s) \sim \text{Poisson}(\lambda t)$, that is $\mathbb{P}(A(s+t) - A(s) = n) = e^{-\lambda t} (\lambda t)^n / n!$.

$S_n = \inf\{t : A(t) \geq n\}$ is the n -th arrival time, and $T_n = S_n - S_{n-1}$ ($S_0 = 0$) is the n -th **interarrival time**.

Theorem 1.11 (Exponential interarrival times [20, §2.2])

A is a Poisson process with rate λ if and only if the interarrival times $T_1, T_2, \dots$ are independent and each is $\text{Exp}(\lambda)$.

Proof ($\Rightarrow$). For $0 = t_0 < t_1 < \dots < t_n$ and small $h > 0$ consider the event $E_h = \{S_j \in (t_j, t_j + h], j = 1, \dots, n\}$. Let F_h be the event that there is no arrival in $(t_{j-1} + h, t_j]$ and exactly one in each $(t_j, t_j + h]$. Then $F_h \subset E_h$, and $E_h \setminus F_h$ is contained in the event that some interval of length h has two or more arrivals, which has probability $O(h^{n+1})$. By independent and Poisson increments,

$$\mathbb{P}(F_h) = e^{-\lambda(t_1 + (t_2 - t_1 - h) + \dots + (t_n - t_{n-1} - h))} (\lambda h e^{-\lambda h})^n = \lambda^n h^n e^{-\lambda t_n} (1 + O(h)).$$

Hence $(S_1, \dots, S_n)$ has density $\lambda^n e^{-\lambda s_n}$ on $\{0 < s_1 < \dots < s_n\}$. The linear map $T_j = S_j - S_{j-1}$ has Jacobian one and $s_n = \sum_j T_j$, so $(T_1, \dots, T_n)$ has density $\prod_{j=1}^n \lambda e^{-\lambda t_j}$, that is, independent $\text{Exp}(\lambda)$. The converse is a standard argument using that S_n is Gamma distributed together with memorylessness, and we omit it [20, §2.2]. $\square$

Proposition 1.12 (Superposition and thinning [10])

- (i) **(Superposition)** If A_1, A_2 are independent Poisson processes with rates λ_1, λ_2 , then $A_1 + A_2$ is a Poisson process with rate $\lambda_1 + \lambda_2$.
- (ii) **(Thinning)** If each point of a Poisson process with rate λ is kept with probability p and discarded with probability $1 - p$, independently of everything else, then the kept points and the discarded points are Poisson processes with rates $p\lambda$ and $(1 - p)\lambda$, and they are independent.

Proof. (i) Independent increments follow from the independent increments of the two processes and their mutual independence. For the increment over one interval, the generating function of independent Poisson variables is $\mathbb{E}[z^{X_1+X_2}] = e^{\lambda_1 t(z-1)} e^{\lambda_2 t(z-1)} = e^{(\lambda_1+\lambda_2)t(z-1)}$, so the sum is $\text{Poisson}((\lambda_1 + \lambda_2)t)$.

(ii) Let X be the original increment over an interval of length t , X_1 the number kept, and $X_2 = X - X_1$ the number discarded. Given $X = j + k$, the conditional probability that $X_1 = j$ is binomial, $\binom{j+k}{j} p^j (1-p)^k$, so

$$\mathbb{P}(X_1 = j, X_2 = k) = e^{-\lambda t} \frac{(\lambda t)^{j+k}}{(j+k)!} \binom{j+k}{j} p^j (1-p)^k = e^{-p\lambda t} \frac{(p\lambda t)^j}{j!} \cdot e^{-(1-p)\lambda t} \frac{((1-p)\lambda t)^k}{k!}.$$

Thus on one interval X_1, X_2 are independent Poisson variables. On disjoint intervals the original increments and the coin flips are all independent, so all increments of the two processes are independent. $\square$

In LLM serving

If sessions arrive as a Poisson process (rate Λ in the paper) and each turn is assigned independently to one of J replicas, then by Proposition 1.12(ii) each replica also sees Poisson arrivals. The turns within a session, however, go around a closed loop (prefill $\rightarrow$ decode $\rightarrow$ tool), so the turn arrivals are not exactly Poisson. The closed models of Lecture 4 are the tool for this gap.

1.5 PASTA: Poisson arrivals see time averages

The distribution of the state seen by arriving customers generally differs from the time-average distribution. For example, in a D/D/1 system with interarrival times exactly 1 and service times exactly 0.5, the server is busy half of the time, yet every arriving customer finds it idle. With Poisson arrivals this discrepancy disappears.

Definition 1.13 (Lack of anticipation)

Given a state process $X = (X(t))_{t \geq 0}$ and an arrival process A , we say that A has **lack of anticipation** with respect to X if for every t the future increments $\{A(t+s) - A(t) : s \geq 0\}$ are independent of the past $\{X(u) : u \leq t\} \cup \{A(u) : u \leq t\}$.

In a queueing system this assumption says that future arrivals are independent of the system state so far. Note the direction: the state may (and does) depend on past arrivals; what is excluded is that future arrivals react to the state. Poisson arrivals from outside satisfy it naturally. Arrivals in a closed system (Lecture 4) do not.

Example 1.14 (One process with lack of anticipation, one without)

(a) *Lack of anticipation: the M/M/1 queue.* Customers come from a large outside population as a Poisson process, independently of the service times. The queue length $X(t)$ is a function of the arrivals before t and the service times, and by independent increments the arrivals after t are independent of both. An arriving customer is like a raindrop falling on the queue: the rain does not know how long the queue is. So the arrival instants are “random inspection times”, and by Theorem 1.16 the fraction of arrivals that find the server busy equals the fraction of time it is busy, ρ .

(b) *No lack of anticipation: a single agent session.* One session alternates between a tool call (thinking, mean Z) and a turn at the server (mean $1/\mu$). Its next turn can arrive only after its previous turn has left the server. If $X(t) = 1$ (the turn is being served), then no arrival can happen until the service ends; if $X(t) = 0$, an arrival may come at any moment. Future arrivals depend on the current state, so the condition fails. The consequence is visible: every arrival finds the server idle, while the server is busy a fraction $(1/\mu)/(Z + 1/\mu) > 0$ of the time. Arrivals do not see time averages. With N sessions the same effect is weaker but present, and Lecture 4 replaces PASTA by the arrival theorem for exactly this reason.

(c) *Lack of anticipation is not enough.* In the D/D/1 example above, the arrivals at times $1, 2, 3, \dots$ are deterministic, hence independent of everything, so they have lack of anticipation. Yet every arrival finds the server idle. What fails is the other hypothesis of Theorem 1.16: the arrivals are not Poisson. They are locked in phase with the service, always landing after the previous customer has left. Poisson arrivals are spread uniformly over time and cannot lock onto any phase.

Remark 1.15 (Why study PASTA if sessions violate it?)

Example 1.14(b) shows that a single agent session does not have lack of anticipation. PASTA is still the right starting point, for three reasons.

1. *The prefill model rests on it.* The paper models the prefill stage as an M/G/1 queue, and the proof of the Pollaczek–Khinchine formula (Theorem 3.5) uses PASTA. The paper states that session feedback makes this an approximation and tests it. One cannot judge an approximation without knowing what it approximates.
2. *Many sessions look Poisson.* Each session reacts to the state, but the superposition of many independent sessions, each contributing a small share of the arrivals, is close to a Poisson process; this is the Palm–Khinchine theorem [9]. New sessions themselves arrive as a Poisson process in the model, so the M/G/ ∞ and Campbell calculations of Lecture 4 use Poisson arrivals without approximation.
3. *The exact closed-system answer is a corrected PASTA.* The arrival theorem (Theorem 4.3) says that an arriving turn sees the time averages of the system with one session fewer, and it reduces to PASTA as the number of sessions grows (Remark 4.4). For exponential work, Proposition 4.6(ii) shows that the open formula built on PASTA overstates the wait at equal utilisation, so PASTA gives a conservative estimate whose error the finite-population model quantifies.

Theorem 1.16 (PASTA [23])

Let A be a Poisson process with rate λ , X a state process with left limits, and B a set of states, and suppose A has lack of anticipation with respect to X . If $U(t) = \mathbf{1}\{X(t^-) \in B\}$ is left-continuous, then for every $T > 0$

$$\mathbb{E}\left[\sum_{k: S_k \leq T} \mathbf{1}\{X(S_k^-) \in B\}\right] = \lambda \mathbb{E}\left[\int_0^T \mathbf{1}\{X(t) \in B\} dt\right]. \quad (1.2)$$

Consequently, if the limits of the two averages $\frac{1}{T} \int_0^T \mathbf{1}\{X(t) \in B\} dt$ and $\frac{1}{A(T)} \sum_{k \leq A(T)} \mathbf{1}\{X(S_k^-) \in B\}$ exist with probability one, they are equal: the fraction of arrivals that find the system in B equals the fraction of time it spends in B .

Proof. The left side is $\int_0^T U(t) dA(t)$. For the partition $t_j = jT/n$ of $[0, T]$, define the step function $U_n(t) = \sum_j U(t_j) \mathbf{1}\{t \in (t_j, t_{j+1}]\}$. $U(t_j)$ is determined by the past up to time t_j , and by lack of anticipation the increment $A(t_{j+1}) - A(t_j)$ is independent of that past, so

$$\mathbb{E}\left[\int_0^T U_n dA\right] = \sum_j \mathbb{E}[U(t_j)] \mathbb{E}[A(t_{j+1}) - A(t_j)] = \lambda \sum_j \mathbb{E}[U(t_j)] (t_{j+1} - t_j) = \lambda \mathbb{E}\left[\int_0^T U_n(t) dt\right].$$

Since U is left-continuous, $U_n(t) \rightarrow U(t)$ for every t . The integrands on the two sides are bounded by $A(T)$ and T respectively, and $\mathbb{E}[A(T)] = \lambda T < \infty$, so by dominated convergence $\mathbb{E}[\int_0^T U dA] = \lambda \mathbb{E}[\int_0^T U dt]$ as $n \rightarrow \infty$. Finally, the set of t with $X(t^-) \neq X(t)$ is countable, so replacing $U(t)$ by $\mathbf{1}\{X(t) \in B\}$ on the right does not change the integral. The statement about limits follows from a sample-path version of (1.2) (a law of large numbers applied to a martingale), for which we cite [23]. $\square$

In Lecture 3, PASTA becomes a key step of the Pollaczek–Khinchine formula in the form “the mean work an arriving turn finds waiting equals the time-average work waiting”.

1.6 The renewal-reward theorem

Definition 1.17 (Renewal process and rewards)

For i.i.d. positive random variables $C_1, C_2, \dots$ with $\mathbb{E}[C_1] < \infty$, let $\tau_n = C_1 + \dots + C_n$ and $M(t) = \#\{n \geq 1 : \tau_n \leq t\}$. M is a **renewal process** and C_n is its n -th **cycle**. Suppose each cycle carries a reward $R_n \geq 0$, the pairs (C_n, R_n) are i.i.d., and the cumulative reward $R(t)$ up to time t satisfies

$$\sum_{n=1}^{M(t)} R_n \leq R(t) \leq \sum_{n=1}^{M(t)+1} R_n$$

(this includes rewards that accrue gradually within a cycle). This is a **renewal-reward process**.

Theorem 1.18 (Renewal-reward theorem [20])

If $\mathbb{E}[R_1] < \infty$, then with probability one $\lim_{t \rightarrow \infty} \frac{R(t)}{t} = \frac{\mathbb{E}[R_1]}{\mathbb{E}[C_1]}$.

Proof. First we show $M(t)/t \rightarrow 1/\mathbb{E}[C_1]$ a.s. By definition $\tau_{M(t)} \leq t < \tau_{M(t)+1}$, and since each

$\tau_n < \infty$, $M(t) \rightarrow \infty$. By the strong law, $\tau_n/n \rightarrow \mathbb{E}[C_1]$, so both ends of

$$\frac{\tau_{M(t)}}{M(t)} \leq \frac{t}{M(t)} < \frac{\tau_{M(t)+1}}{M(t)+1} \cdot \frac{M(t)+1}{M(t)}$$

converge to $\mathbb{E}[C_1]$. Now

$$\frac{1}{t} \sum_{n=1}^{M(t)} R_n = \frac{M(t)}{t} \cdot \frac{1}{M(t)} \sum_{n=1}^{M(t)} R_n \rightarrow \frac{1}{\mathbb{E}[C_1]} \mathbb{E}[R_1]$$

(the strong law for the second factor), and the upper bound has the same limit since $(M(t)+1)/t \rightarrow 1/\mathbb{E}[C_1]$. The conclusion follows by squeezing. $\square$

In the form “long-run time average = expected reward per cycle / expected cycle length”, the renewal-reward theorem is used directly for the residual service time (the inspection paradox) in Lecture 3.

1.7 Little’s law

Theorem 1.19 (Little’s law [14, 22])

For a sequence of customers (a_k, d_k) with $a_k \leq d_k < \infty$ and a_k nondecreasing, suppose the two limits

$$\lambda = \lim_{t \rightarrow \infty} \frac{A(t)}{t} \in (0, \infty), \quad W = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n W_k < \infty$$

exist. Then $L = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t N(s) ds$ exists and

$$L = \lambda W. \quad (1.3)$$

Proof. Since $N(s) = \sum_k \mathbf{1}\{a_k \leq s < d_k\}$, we have $\int_0^t N(s) ds = \sum_k |[a_k, d_k) \cap [0, t]|$. Each term is at most W_k , is zero if $a_k > t$, and equals W_k exactly if $d_k \leq t$. Therefore

$$\sum_{k: d_k \leq t} W_k \leq \int_0^t N(s) ds \leq \sum_{k: a_k \leq t} W_k = \sum_{k=1}^{A(t)} W_k. \quad (1.4)$$

Upper bound: since $A(t) \rightarrow \infty$, $\frac{1}{t} \sum_{k \leq A(t)} W_k = \frac{A(t)}{t} \cdot \frac{1}{A(t)} \sum_{k \leq A(t)} W_k \rightarrow \lambda W$.

Lower bound: with $\bar{W}_n = \frac{1}{n} \sum_{k \leq n} W_k$ we have $\frac{W_n}{n} = \bar{W}_n - \frac{n-1}{n} \bar{W}_{n-1} \rightarrow W - W = 0$. Also $A(a_n) \geq n$ and $A(a_n^-) \leq n-1$, so $a_n/n \rightarrow 1/\lambda$, and therefore $W_n/a_n \rightarrow 0$. For any $\varepsilon > 0$ there is k_0 such that $d_k = a_k + W_k \leq (1+\varepsilon)a_k$ for $k \geq k_0$. Then for $t \geq \max_{k < k_0} d_k$, every customer with $a_k \leq t/(1+\varepsilon)$ has $d_k \leq t$, so

$$\frac{1}{t} \int_0^t N(s) ds \geq \frac{1}{t} \sum_{k=1}^{A(t/(1+\varepsilon))} W_k \rightarrow \frac{\lambda W}{1+\varepsilon}.$$

Letting $\varepsilon \downarrow 0$, the liminf is at least λW . Together with the upper bound this gives (1.3). $\square$

The proof uses no distributional assumption and no service discipline. It is a deterministic theorem that holds on every sample path on which the limits exist. So depending on how we draw the boundary of “the system”, we obtain several identities.

Corollary 1.20 (Utilisation law [12])

Under the assumptions of Theorem 1.19, consider a single-server system and suppose the average of the service times $\frac{1}{n} \sum_{k \leq n} S_k$ converges to $\mathbb{E}[S]$.

- (i) The fraction of time the server is busy is $\rho = \lambda \mathbb{E}[S]$. In particular $\rho \leq 1$ in a stable system.
- (ii) For the waiting room, $L_q = \lambda W_q$, and under FIFO $L = L_q + \rho$.

Proof. (i) Take “the server” alone as the system. Customer k enters it when its service starts and leaves S_k later. Every customer is eventually served, so the arrival rate of this subsystem is also λ , and the customer average of its sojourn times is $\mathbb{E}[S]$. The number in this subsystem is 1 when the server is busy and 0 otherwise, so its time average is the fraction of time the server is busy. By Theorem 1.19 this equals $\lambda \mathbb{E}[S] = \rho$, and being a fraction it is at most one. (ii) Taking the waiting room alone as the system, the sojourn times are $W_{q,k}$, so $L_q = \lambda W_q$. Taking time averages in $N = N_q + (\text{number in service})$ gives $L = L_q + \rho$. $\square$

Example 1.21

Suppose $\lambda = 2$ turns per second arrive at the prefill stage and the mean prefill work is $\mathbb{E}[S] = 0.3$ seconds. By Corollary 1.20 the server is busy a fraction $\rho = 0.6$ of the time. If the observed mean number of waiting turns is $L_q = 0.9$, then $W_q = L_q/\lambda = 0.45$ seconds, the mean TTFT is $W_q + \mathbb{E}[S] = 0.75$ seconds, and the mean number of turns in the stage is $L = \lambda \cdot 0.75 = 1.5$.

The paper takes as its objective the mean number of turns in the prefill stage and in the decode batch, $L = L_P + L_D$. By Theorem 1.19 this equals the total delay accumulated per unit time, $\lambda \cdot (\text{mean sojourn time})$, so “reduce the mean number of turns” and “reduce the mean delay” are the same goal. The “price of a miss” of Lecture 3 is defined precisely as a change in this L_P .

1.8 Exercises

Exercise 1.1. For independent $X_1 \sim \text{Exp}(\mu_1)$ and $X_2 \sim \text{Exp}(\mu_2)$, show that $\mathbb{P}(X_1 < X_2) = \mu_1/(\mu_1 + \mu_2)$, and that the event $\{X_1 < X_2\}$ is independent of $\min(X_1, X_2)$. *Hint:* integrate $\mathbb{P}(X_1 < X_2, \min > t)$ against the density of X_1 .

Exercise 1.2. Each time a session makes a tool call, it stays away from the server for a mean of Z seconds. If tool calls start at rate λ , show that the time-average number of sessions in a tool call is λZ . Does this hold whatever the distribution of the tool-call duration? *Hint:* take the tool-call stage alone as “the system” and apply Theorem 1.19.

Lecture 2

Markovian Queues: M/M/1, Processor Sharing, and Insensitivity

Lecture 1 dealt with facts, such as the arrival process and Little's law, that depend hardly at all on the details of the model. This lecture treats the case where the state of the system (the number of jobs present) is a continuous-time Markov chain. The stationary distribution can then be computed exactly, and the model of the paper's decode stage (processor sharing) and its key property, *insensitivity*, come out of it.

2.1 Continuous-time Markov chains

Definition 2.1 (Continuous-time Markov chain)

A **generator** (Q -matrix) on a countable set $\mathcal{X}$ is a matrix $Q = (q(x, y))_{x, y \in \mathcal{X}}$ with $q(x, y) \geq 0$ for $x \neq y$, $q(x) := \sum_{y \neq x} q(x, y) < \infty$, and $q(x, x) = -q(x)$. A right-continuous process $(X_t)_{t \geq 0}$ with values in $\mathcal{X}$ is a **continuous-time Markov chain** (CTMC) with generator Q if, on entering state x , it stays there for an exponential time with mean $1/q(x)$ (independent of the past) and then jumps to y with probability $q(x, y)/q(x)$. $q(x, y)$ is the **transition rate** from x to y . The chain is **non-explosive** if infinitely many jumps in finite time have probability zero, and **irreducible** if every y can be reached from every x with positive probability.

The memorylessness of the exponential distribution (Proposition 1.8) is what makes this definition possible. If holding times were not exponential, how long the chain has already stayed would affect the future and the Markov property would fail. Intuitively, $q(x, y)$ means that the probability of moving from x to y in a short time h is $q(x, y)h + o(h)$.

Definition 2.2 (Stationary distribution, balance equations)

A probability distribution π on $\mathcal{X}$ with $\pi Q = 0$, that is, for all x ,

$$\pi(x) q(x) = \sum_{y \neq x} \pi(y) q(y, x), \quad (2.1)$$

is a **stationary distribution**, and (2.1) are the **global balance equations**. The left side is the probability flow out of x , the right side the probability flow into x . The stronger condition

$$\pi(x) q(x, y) = \pi(y) q(y, x) \quad (\forall x \neq y) \quad (2.2)$$

is called **detailed balance**.

Lemma 2.3 (Detailed balance [17])

A probability distribution satisfying detailed balance is stationary.

Proof. Summing (2.2) over $y \neq x$ gives $\pi(x) \sum_{y \neq x} q(x, y) = \sum_{y \neq x} \pi(y) q(y, x)$, which is (2.1). $\square$

We quote the next theorem without proof [17, Ch. 3]. It lets us read the stationary probability $\pi(x)$ as “the fraction of time the state is x ”.

Theorem 2.4 (Ergodic theorem [17])

If an irreducible, non-explosive CTMC has a stationary distribution π , then π is unique, the chain is positive recurrent, and for any initial state and any f with $\sum_x |f(x)|\pi(x) < \infty$, almost surely

$$\frac{1}{t} \int_0^t f(X_s) ds \rightarrow \sum_x f(x)\pi(x) \quad (t \rightarrow \infty).$$

2.2 Birth–death processes and M/M/1

Definition 2.5 (Birth–death process)

A CTMC on $\mathcal{X} = \{0, 1, 2, \dots\}$ whose only nonzero transition rates are $q(n, n+1) = \lambda_n > 0$ ($n \geq 0$) and $q(n, n-1) = \mu_n > 0$ ($n \geq 1$) is a **birth–death process**. λ_n are the birth rates and μ_n the death rates.

In a queue, n is the number of jobs in the system. An arrival increases n by one and a completion decreases it by one, so if arrivals and completions happen on exponential clocks, the number of jobs is a birth–death process. It is known that the process is non-explosive when the birth rates are bounded.

Theorem 2.6 (Stationary distribution of a birth–death process [12])

A birth–death process has a stationary distribution if and only if

$$H := \sum_{n=0}^{\infty} \prod_{k=1}^n \frac{\lambda_{k-1}}{\mu_k} < \infty \quad (2.3)$$

(an empty product is 1), and then the stationary distribution is $\pi(n) = H^{-1} \prod_{k=1}^n \lambda_{k-1} / \mu_k$.

Proof. (Necessity and the form of π .) Suppose π satisfies (2.1). At state 0, $\pi(0)\lambda_0 = \pi(1)\mu_1$. For $n \geq 1$, assume $\pi(n-1)\lambda_{n-1} = \pi(n)\mu_n$. Cancelling this term from the global balance at state n , $\pi(n)(\lambda_n + \mu_n) = \pi(n-1)\lambda_{n-1} + \pi(n+1)\mu_{n+1}$, gives $\pi(n)\lambda_n = \pi(n+1)\mu_{n+1}$. By induction, for all n

$$\pi(n)\lambda_n = \pi(n+1)\mu_{n+1}, \quad (2.4)$$

hence $\pi(n) = \pi(0) \prod_{k=1}^n \lambda_{k-1} / \mu_k$. For $\sum_n \pi(n) = 1$ we need $\pi(0) = 1/H > 0$, that is, $H < \infty$. (Sufficiency.) If $H < \infty$, the π defined above is a probability distribution, and (2.4) is exactly detailed balance (2.2), so π is stationary by Lemma 2.3. $\square$

Eq. (2.4) is called the **cut equation**: the flow across the boundary between $\{0, \dots, n\}$ and $\{n+1, \dots\}$ is the same in both directions.

Definition 2.7 (M/M/1 queue)

In Kendall notation, **M/M/1** is the queue with Poisson arrivals of rate λ (Definition 1.10), service times exponential with mean $1/\mu$ (independent of each other and of the arrivals), one server, and unlimited waiting room. Its load is $\rho = \lambda/\mu$ (Definition 1.6).

Proposition 2.8 (M/M/1 [12])

In M/M/1, the number of jobs N_t (including the one in service) is a birth–death process with $\lambda_n = \lambda$, $\mu_n = \mu$. If $\rho < 1$, the stationary distribution is geometric, $\pi(n) = (1 - \rho)\rho^n$, and

$$L = \frac{\rho}{1 - \rho}, \quad W = \frac{1}{\mu - \lambda}, \quad W_q = \frac{\rho}{\mu(1 - \rho)}.$$

If $\rho \geq 1$, there is no stationary distribution. As $\rho \uparrow 1$, L , W and W_q all diverge to $+\infty$.

Proof. When $N_t = n \geq 1$, by memorylessness the time to the next arrival is $\text{Exp}(\lambda)$, the residual service time of the job in service is $\text{Exp}(\mu)$, and the two are independent. So the rate of $n \rightarrow n + 1$ is λ and that of $n \rightarrow n - 1$ is μ . In Theorem 2.6, $H = \sum_n \rho^n$ is finite only when $\rho < 1$, and then $H = 1/(1 - \rho)$. Hence $\pi(n) = (1 - \rho)\rho^n$ and $L = \sum_n n(1 - \rho)\rho^n = \rho/(1 - \rho)$. By Little’s law (Theorem 1.19), $W = L/\lambda = 1/(\mu - \lambda)$, and since the waiting time is the sojourn time minus the service time, $W_q = W - 1/\mu = \rho/(\mu(1 - \rho))$. $\square$

At $\rho = 0.9$ we get $L = 9$, and at $\rho = 0.99$, $L = 99$. This factor $1/(1 - \rho)$, which says that near capacity a small increase in mean load greatly increases the delay, reappears in the “price of a miss” of Lecture 3.

2.3 Processor sharing and state-dependent capacity

So far the server has processed one job at a time. The decode stage of an LLM does not.

Definition 2.9 (Work, processor sharing)

The **work** (service requirement) $S \geq 0$ of a job is the time the server needs to process it at full speed. Given a **capacity function** $\varphi : \{1, 2, \dots\} \rightarrow (0, \infty)$, the rule under which, when n jobs are present, the server processes $\varphi(n)$ units of work per second and splits it equally among the n jobs, so that each job’s remaining work decreases at rate $\varphi(n)/n$, is called **processor sharing** (PS). $\varphi \equiv 1$ is classical PS; with Poisson arrivals and a general work distribution G we write **M/G/1-PS**. The load is $\rho = \lambda\mathbb{E}[S]$.

In LLM serving

In continuous batching, one decode step advances every request in the batch by one token. We therefore model the decode stage as PS, with the m requests sharing the device equally. Because the weights are read once for the whole batch, $\varphi(m)$ increases with m , and because memory (the KV cache) limits the batch size, it eventually saturates. The decode stage of §2.2 of the paper is exactly this model: the work is the decode demand D_i and the load is $\rho_D = \lambda\mathbb{E}[D]$.

Proposition 2.10 (PS with exponential work [7])

For a PS server with capacity φ , Poisson arrivals of rate λ , and exponential work with mean $\mathbb{E}[S]$, the number of jobs is a birth–death process with $\lambda_n = \lambda$ and $\mu_n = \varphi(n)/\mathbb{E}[S]$. Hence, with $F(n) := \prod_{k=1}^n \varphi(k)$, if $G(\rho) := \sum_{n \geq 0} \rho^n / F(n) < \infty$ then

$$\pi(n) = \frac{\rho^n / F(n)}{G(\rho)}, \quad \rho = \lambda \mathbb{E}[S]. \quad (2.5)$$

Proof. In state n , by memorylessness the remaining works of the n jobs are independent, each $\text{Exp}(1/\mathbb{E}[S])$, and each decreases at speed $\varphi(n)/n$. Hence each job’s completion time is exponential with rate $\varphi(n)/(n\mathbb{E}[S])$, and the first of the n completions is exponential with rate $\varphi(n)/\mathbb{E}[S]$ (Lemma 1.9). The rest follows from Theorem 2.6, since $\prod_{k \leq n} \lambda \mathbb{E}[S] / \varphi(k) = \rho^n / F(n)$. $\square$

With $\varphi \equiv 1$, (2.5) is the geometric distribution of M/M/1. With exponential work, the service order does not change the distribution of the number of jobs.

Example 2.11 (Saturating capacity)

Let $\varphi(m) = \min(m, c)$. Below c the throughput grows in proportion to the batch size, and it saturates at c (the simplest model of a memory that holds only c requests). This is the same birth–death process as M/M/ c with c servers, and by (2.5), $\pi(n) \propto \rho^n / n!$ for $n \leq c$ and $\pi(n) \propto \rho^n / (c! c^{n-c})$ for $n \geq c$. $G(\rho) < \infty$ if and only if $\rho < c = \sup \varphi$. If the load exceeds the saturated capacity, the system is unstable however large the batch grows.

The next theorem says that more expected demand never leaves fewer jobs. This sounds obvious, but it is not true in general: it depends on the discipline.

More work, fewer jobs. At a single FIFO server compare two work laws. Under law A a job takes 0.1 with probability 0.9 and 9.1 with probability 0.1; under law B every job takes 1.2, so B carries 20% more work. In Figure 2.1 the long job of law A holds the server while short jobs pile up behind it, and the number in system reaches 7; under law B it never exceeds 2. In steady state, with Poisson arrivals at $\lambda = 0.5$ ($\rho = 0.5$ for A, 0.6 for B), the Pollaczek–Khinchine formula of Lecture 3 gives a mean number in system of 2.57 under A and 1.05 under B. Under FIFO the wait is set by the second moment of the work, not its mean, so less expected demand does not mean fewer jobs.

Under processor sharing a short job shares the server with the long one and leaves after about 0.2. With $\varphi \equiv 1$ the mean number in system is $\rho/(1-\rho)$ for every work law (Theorem 2.14 below): 1.0 under A and 1.5 under B, in the order of the load. Theorem 2.12 together with insensitivity makes this true for every capacity function φ .

Theorem 2.12 (Monotonicity of the mean batch size)

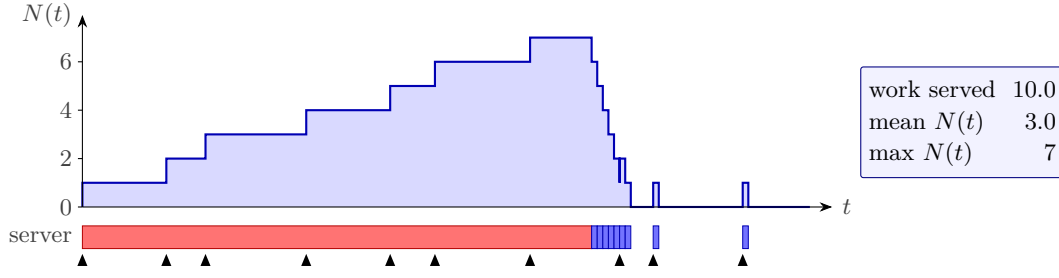
Let φ be any positive function and R the radius of convergence of the power series $G(\rho) = \sum_n \rho^n / F(n)$. The mean $L(\rho) = \sum_n n \pi(n)$ of the stationary distribution (2.5) is differentiable on $(0, R)$ and

$$\frac{dL}{d\rho} = \frac{\text{Var}_\rho(N)}{\rho} \geq 0.$$

Hence L is nondecreasing in the load. If φ is nondecreasing, then $R = \sup_n \varphi(n)$.

Proof. Inside its radius of convergence a power series can be differentiated term by term, and the differentiated series has the same radius (a standard theorem of analysis). With $a_n = 1/F(n)$ we

Law A: short jobs (0.1) with an occasional long one (9.1, probability 0.1)



Law B: every job takes 1.2 (20% more work per job)

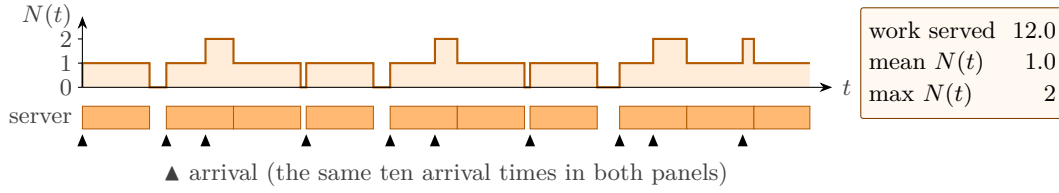


Figure 2.1: One sample path of each work law at a FIFO server, on the same arrival times. Top of each panel: the number in system $N(t)$; bottom: when the server works on each job (red: the long job of law A). *What to see:* law B does 20% more work, yet law A keeps about three times as many jobs waiting, because one long job blocks everything behind it.

have $\rho G'(\rho) = \sum_n n a_n \rho^n$ and $\rho^2 G''(\rho) = \sum_n n(n-1) a_n \rho^n$, so

$$L = \frac{\rho G'}{G}, \quad \mathbb{E}_\rho[N(N-1)] = \frac{\rho^2 G''}{G}.$$

By the quotient rule,

$$\rho \frac{dL}{d\rho} = \frac{\rho G'}{G} + \frac{\rho^2 G''}{G} - \left(\frac{\rho G'}{G} \right)^2 = \mathbb{E}[N] + \mathbb{E}[N(N-1)] - \mathbb{E}[N]^2 = \text{Var}(N) \geq 0.$$

If φ is nondecreasing with $\varphi_\infty := \sup \varphi$, then $a_{n+1}/a_n = 1/\varphi(n+1) \rightarrow 1/\varphi_\infty$, so by the ratio test $R = \varphi_\infty$ ($R = \infty$ if $\varphi_\infty = \infty$). $\square$

For $\rho > 0$, N is not degenerate, so in fact L is strictly increasing. The theorem says, without knowing anything about the shape of φ , that “a policy that increases the mean demand does not shrink the batch”. Since φ must be calibrated from measurements, such shape-free conclusions matter.

2.4 Insensitivity

Proposition 2.10 assumed exponential work. But the decode demand is proportional to the output length, and there is no reason for it to be exponential. Remarkably, under PS the conclusion does not depend on the work distribution.

Definition 2.13 (Insensitivity)

A stationary quantity of a queue (here, the distribution of the number of jobs) is **insensitive** to the work distribution G if it depends on G only through its mean $\int x dG(x)$.

Theorem 2.14 (Insensitivity of PS [7, 2])

For a PS server with capacity φ and Poisson arrivals of rate λ , the stationary distribution of the number of jobs is insensitive to the work distribution and equals (2.5). We prove it here when the work is a finite mixture of exponentials, that is, $\text{Exp}(\mu_k)$ with probability α_k ($k = 1, \dots, K$, $\sum_k \alpha_k = 1$).

Proof. Think of each job as independently drawing a type k with probability α_k on arrival and then drawing its work from $\text{Exp}(\mu_k)$. The vector of job counts by type, $\mathbf{n} = (n_1, \dots, n_K)$ with $n = |\mathbf{n}| = \sum_k n_k$, is a CTMC. By thinning of the Poisson process (Proposition 1.12(ii)), type k arrives at rate $\lambda\alpha_k$ ($\mathbf{n} \rightarrow \mathbf{n} + e_k$), and as in the proof of Proposition 2.10 a single type- k job finishes at rate $\mu_k\varphi(n)/n$, so the rate of $\mathbf{n} \rightarrow \mathbf{n} - e_k$ is $n_k\mu_k\varphi(n)/n$. Let $\rho_k = \lambda\alpha_k/\mu_k$. We show that

$$\pi(\mathbf{n}) = B \frac{n!}{n_1! \cdots n_K!} \frac{\prod_k \rho_k^{n_k}}{F(n)}$$

satisfies global balance. If $n_k \geq 1$, then $\pi(\mathbf{n})/\pi(\mathbf{n} - e_k) = \frac{n}{n_k} \cdot \frac{\rho_k}{\varphi(n)}$, so

$$\pi(\mathbf{n}) \frac{n_k\mu_k\varphi(n)}{n} = \pi(\mathbf{n} - e_k) \rho_k\mu_k = \pi(\mathbf{n} - e_k) \lambda\alpha_k. \quad (2.6)$$

This says “the flow out of $\mathbf{n}$ by a type- k completion equals the flow into $\mathbf{n}$ by a type- k arrival”. Applying (2.6) at $\mathbf{n} + e_k$ and summing over k ,

$$\sum_k \pi(\mathbf{n} + e_k) \frac{(n_k + 1)\mu_k\varphi(n + 1)}{n + 1} = \sum_k \pi(\mathbf{n}) \lambda\alpha_k = \pi(\mathbf{n})\lambda,$$

that is, “the flow out by arrivals equals the flow in by completions”. Adding the two gives global balance at $\mathbf{n}$. Splitting global balance in this way is called **partial balance**. Now, by the multinomial theorem,

$$\mathbb{P}(N = n) = \sum_{|\mathbf{n}|=n} \pi(\mathbf{n}) = \frac{B}{F(n)} \sum_{|\mathbf{n}|=n} \frac{n!}{\prod_k n_k!} \prod_k \rho_k^{n_k} = \frac{B \rho^n}{F(n)}, \quad \rho = \sum_k \rho_k = \lambda \sum_k \frac{\alpha_k}{\mu_k} = \lambda \mathbb{E}[S].$$

The normalisability condition is $G(\rho) < \infty$, with $B = 1/G(\rho)$. This is (2.5), and G involves only $\mathbb{E}[S]$. (Uniqueness follows from irreducibility and Theorem 2.4.) $\square$

Remark 2.15

Insensitivity for general work distributions is due to [7] and [2] (BCMP). One route to a proof is to extend the partial balance argument above to phase-type distributions and take limits, using the fact that phase-type distributions are dense, under weak convergence, among distributions on $[0, \infty)$. Here we only cite it. The same insensitivity holds for a **delay station** (an infinite-server station where every job immediately gets its own server, so nobody waits), which we use in Lecture 4.

FIFO is not insensitive. Compare three service-time distributions with the same mean $\mathbb{E}[S] = 1$ and the same load $\rho = 0.5$ ($\lambda = 0.5$). Borrowing from Lecture 3 the Pollaczek–Khinchine formula $W_q = \lambda \mathbb{E}[S^2]/(2(1 - \rho))$ and Little’s law $L = \lambda W_q + \rho$, we get the following.

Service time S	$\mathbb{E}[S^2]$	FIFO L	PS L
constant 1	1	0.75	1
$\text{Exp}(1)$	2	1.00	1
0.1 w.p. 0.9, 9.1 w.p. 0.1	8.29	2.57	1

Under FIFO one long job blocks every job behind it (head-of-line blocking). Under PS short jobs share capacity with the long one and leave first. The fairness of PS is also visible job by job: in M/G/1-PS, the mean sojourn time of a job with work x is $x/(1 - \rho)$, proportional to its own size and independent of the size distribution of the other jobs [11]. This difference separates the two stages of the paper: the FIFO prefill queue sees the second moment of the work, while the PS decode stage sees only the mean.

2.5 The price at the decode stage

Proposition 2.16 (The decode stage prices only work)

For a PS station with constant capacity C and load $\rho < C$, $L = \rho/(C - \rho)$. If a policy change increases the work of a fraction q_i of arrivals by $\Delta D_i \geq 0$, raising the load to $\rho' = \rho + \lambda \sum_i q_i \Delta D_i < C$, then exactly

$$\Delta L = \frac{C(\rho' - \rho)}{(C - \rho)(C - \rho')} = \frac{C - \rho}{C - \rho'} \lambda \sum_i q_i \Phi_i^D, \quad \Phi_i^D = \frac{C \Delta D_i}{(C - \rho)^2}.$$

In particular $\lambda \sum_i q_i \Phi_i^D \leq \Delta L$.

Proof. With $\varphi \equiv C$ we have $F(n) = C^n$, and (2.5) is geometric with ratio ρ/C , so $L = (\rho/C)/(1 - \rho/C) = \rho/(C - \rho)$. Therefore

$$\Delta L = \frac{\rho'}{C - \rho'} - \frac{\rho}{C - \rho} = \frac{\rho'(C - \rho) - \rho(C - \rho')}{(C - \rho)(C - \rho')} = \frac{C(\rho' - \rho)}{(C - \rho)(C - \rho')}.$$

Substituting $\rho' - \rho = \lambda \sum_i q_i \Delta D_i$ and multiplying and dividing by $(C - \rho)^2$ gives the second equality. Since $\rho' \geq \rho$, the leading factor is at least one. $\square$

$\Phi_i^D = \Delta D_i L'(\rho)$ is the first-order approximation, and the factor $(C - \rho)/(C - \rho')$ corrects for curvature. For example, with $C = 1$ and $\rho = 0.5 \rightarrow 0.6$, $\Delta L = 0.5$, the first-order term is 0.4, and the factor is 1.25.

Corollary 2.17

If a policy change alters the distribution of the decode work but not its mean $\mathbb{E}[D]$, the distribution of the number of jobs at the decode stage does not change.

Proof. By Theorem 2.14 the distribution depends only on $\rho_D = \lambda \mathbb{E}[D]$. $\square$

In LLM serving

KV-cache hits and misses change only the prefill work; they do not change the decode demand $D_i = o_i(\beta K_i + \omega/m)$. So by Corollary 2.17 a miss costs nothing at the decode stage to first order, and its whole price is paid at the FIFO prefill queue (Lecture 3). Only decisions that add decode demand, such as admitting a session, pay the load factor of Proposition 2.16. Proposition 2.16 and Corollary 2.17 are the content of the paper's proposition "The decode stage prices only work".

2.6 Exercises

Exercise 2.1. For $\varphi(n) = n$ (each job has its own server, M/M/ ∞), show that (2.5) is the Poisson distribution with mean ρ , and check the formula $dL/d\rho = \text{Var}(N)/\rho$ of Theorem 2.12 directly. *Hint:* $F(n) = n!$, and a Poisson distribution has equal mean and variance.

Exercise 2.2. Show that in M/M/1-FIFO the sojourn time of an arriving job is $\text{Exp}(\mu - \lambda)$. *Hint:* by PASTA (Theorem 1.16) the number found on arrival is distributed as $(1 - \rho)\rho^n$, and conditionally the sojourn time is a sum of $n + 1$ independent $\text{Exp}(\mu)$ variables.

Exercise 2.3 (M/M/ m and the Erlang C formula). Arrivals are Poisson with rate λ , service times are $\text{Exp}(\mu)$, and there are m servers; a customer waits only when all m are busy. With $a = \lambda/\mu < m$, show from Theorem 2.6 that $\pi(n) = \pi(0) a^n/n!$ for $n \leq m$ and $\pi(n) = \pi(0) a^n/(m! m^{n-m})$ for $n \geq m$, that the probability of waiting is

$$C(m, a) = \frac{\frac{a^m}{m!} \frac{m}{m-a}}{\sum_{k=0}^{m-1} \frac{a^k}{k!} + \frac{a^m}{m!} \frac{m}{m-a}},$$

and that the mean wait is $W_q = C(m, a)/(m\mu - \lambda)$. This is the memory-slot station of Section 1.2 when every turn holds one slot for an exponential time. *Hint:* the death rate in state n is $\min(n, m)\mu$; use PASTA for the waiting probability and Little's law for W_q .

Lecture 3

The M/G/1 Queue and the Pollaczek–Khinchine Formula: the Price of a Miss

In Lecture 2 the service times were exponential, so the state of the queue was summarised by a single number, the number of customers. Prefill work is not exponential. It is a general distribution that mixes short appends (hits) with long recomputations (misses). The goal of this lecture is to compute the mean waiting time of a FIFO queue with a general service distribution (the Pollaczek–Khinchine formula), and then to differentiate it to prove the central proposition of the paper, the price of a miss. The key message is a single one. *The mean wait of a FIFO queue depends not only on the mean of the work but also on its second moment.*

3.1 The M/G/1 queue and the residual service time

Definition 3.1 (M/G/1 FIFO queue)

An **M/G/1 queue** is a single-server queue with the following properties.

- (a) Arrivals form a Poisson process of rate $\lambda > 0$ (Definition 1.10).
- (b) The **service time** (work) S_k of the k -th customer is i.i.d. with an arbitrary distribution G (“General”) and independent of the arrival process. S denotes a generic random variable with law G , and we assume $\mathbb{E}[S] < \infty$.
- (c) There is one server, the waiting room is infinite, and the service discipline is **FIFO** (first in, first out): when the server becomes free, it serves the earliest waiting customer to completion without interruption.

The load is $\rho = \lambda \mathbb{E}[S]$ (Definition 1.6). The **waiting time in queue** W_q of a customer is the time from arrival to the start of service, and the **sojourn time** is $W_q + S$. L_q and L are the time averages of the number of customers in the waiting room and in the whole system (waiting room and server), respectively.

Unlike the exponential distribution, a general service distribution is not memoryless (Proposition 1.8). How much longer a customer who has already been served for 3 seconds will take depends on the information “3 seconds”. The wait of an arriving customer therefore requires the quantity “how much service is left for the customer now in service”.

Definition 3.2 (Residual service time)

If the server is serving a customer at time t , let $R(t)$ be the remaining service time of that customer; if the server is idle, set $R(t) = 0$. $R(t)$ is called the **residual service time**.

The sample path of $R(t)$ is a sawtooth. At the moment service of customer k begins, it jumps up to S_k ; it then decreases with slope -1 , and when it reaches 0 the next service begins or the server becomes idle. Hence the contribution of a single customer k to $\int R(t) dt$ is the area $S_k^2/2$ of a triangle with base and height S_k .

Lemma 3.3 (Time average of the residual service time [12])

In an M/G/1 queue, suppose $\mathbb{E}[S^2] < \infty$ and that the number $D(T)$ of customers who have completed service by time T satisfies $D(T)/T \rightarrow \lambda$ (a.s.) (stability; this holds if $\rho < 1$). Then almost surely

$$\lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T R(t) dt = \frac{\lambda \mathbb{E}[S^2]}{2} = \rho \cdot \frac{\mathbb{E}[S^2]}{2\mathbb{E}[S]}.$$

In particular, averaged only over the time the server is busy, the mean residual service time is $\mathbb{E}[S^2]/(2\mathbb{E}[S])$.

Proof. Let $A(T)$ be the number of arrivals in $[0, T]$. The triangles of the customers who completed service in $[0, T]$ lie entirely in $[0, T]$, and every customer whose service has started arrived before T , so

$$\sum_{k=1}^{D(T)} \frac{S_k^2}{2} \leq \int_0^T R(t) dt \leq \sum_{k=1}^{A(T)} \frac{S_k^2}{2}.$$

(Under FIFO the service order equals the arrival order, so the first $D(T)$ customers are exactly those who have completed service.) By the strong law of large numbers $\frac{1}{n} \sum_{k \leq n} S_k^2 \rightarrow \mathbb{E}[S^2]$; for a Poisson process $A(T)/T \rightarrow \lambda$, and by assumption $D(T)/T \rightarrow \lambda$. Hence

$$\frac{1}{T} \sum_{k=1}^{D(T)} \frac{S_k^2}{2} = \frac{D(T)}{T} \cdot \frac{1}{D(T)} \sum_{k=1}^{D(T)} \frac{S_k^2}{2} \rightarrow \lambda \cdot \frac{\mathbb{E}[S^2]}{2},$$

and the upper sum has the same limit. The squeeze gives the first equality. The second equality is $\lambda = \rho/\mathbb{E}[S]$. The fraction of time the server is busy is ρ (Corollary 1.20) and $R = 0$ during idle time, so the average over busy time is $\lambda \mathbb{E}[S^2]/(2\rho) = \mathbb{E}[S^2]/(2\mathbb{E}[S])$. $\square$

Remark 3.4 (Inspection paradox)

$\mathbb{E}[S^2]/(2\mathbb{E}[S]) = \frac{\mathbb{E}[S]}{2}(1 + \text{CV}^2)$, where $\text{CV}^2 = \text{Var}[S]/\mathbb{E}[S]^2$ is the **squared coefficient of variation**. The intuition “a customer in service has, on average, about half of its service left” fails because, when we look at the server at a random time, the probability of catching a long service is proportional to its length. Indeed, the service observed at a busy instant has the **length-biased distribution** with density $x G(dx)/\mathbb{E}[S]$, whose mean is $\mathbb{E}[S^2]/\mathbb{E}[S]$; the observation instant is uniform within that service, so the mean remaining time is half of it. This is the **inspection paradox**. For the exponential distribution $\text{CV}^2 = 1$, so the mean residual equals $\mathbb{E}[S]$ itself, in agreement with memorylessness.

3.2 The Pollaczek–Khinchine formula

Theorem 3.5 (Pollaczek–Khinchine mean formula [18, 8])

In an M/G/1 FIFO queue let $\rho < 1$ and $\mathbb{E}[S^2] < \infty$. Then in steady state

$$\mathbb{E}[W_q] = \frac{\lambda \mathbb{E}[S^2]}{2(1 - \rho)}, \quad L = \rho + \frac{\lambda^2 \mathbb{E}[S^2]}{2(1 - \rho)}. \quad (3.1)$$

Steady-state assumption. The proof takes the following facts as given. If $\rho < 1$, the queue is stable, the waiting time $W_q^{(k)}$ of the k -th customer converges in distribution, and the limit law has a finite mean (when $\mathbb{E}[S^2] < \infty$). Moreover the customer averages and the time averages exist almost surely. These are standard results obtained by analysing the Lindley recursion $W_q^{(k+1)} = (W_q^{(k)} + S_k - \tau_{k+1})^+$ (τ the interarrival times); see [1, Ch. 3, 10] and [12, Ch. 5]. The proof below accepts this existence and computes the *value* of the limit.

Proof. Consider a customer who has just arrived in steady state (the “tagged customer”). Under FIFO this customer waits for (1) the residual service time of the customer in service and (2) the service times of the N_q customers already in the waiting room:

$$W_q = R + \sum_{j=1}^{N_q} S_{(j)},$$

where R is the residual service time at the arrival instant and $S_{(j)}$ is the service time of the j -th waiting customer.

Mean of (1). Poisson arrivals see time averages (PASTA, Theorem 1.16). So the law of R seen at the arrival instant is the time-stationary law of $R(t)$, and by Lemma 3.3 $\mathbb{E}[R] = \lambda \mathbb{E}[S^2]/2$.

Mean of (2). The waiting customers have not started service yet. Under FIFO the state of the queue up to the arrival of the tagged customer is determined only by the arrival times and the service times of the customers who have already started service. Hence the service times $S_{(1)}, S_{(2)}, \dots$ of the customers not yet started can be regarded as i.i.d. G random variables independent of N_q . Therefore $\mathbb{E}[\sum_{j \leq N_q} S_{(j)}] = \sum_n \mathbb{P}(N_q = n) n \mathbb{E}[S] = \mathbb{E}[N_q] \mathbb{E}[S]$. By PASTA again, $\mathbb{E}[N_q]$ equals the time average L_q of the number in the waiting room.

Combining. Applying Little’s law (Theorem 1.19) to the waiting room alone as the “system” gives $L_q = \lambda \mathbb{E}[W_q]$. Therefore

$$\mathbb{E}[W_q] = \frac{\lambda \mathbb{E}[S^2]}{2} + \lambda \mathbb{E}[W_q] \mathbb{E}[S] = \frac{\lambda \mathbb{E}[S^2]}{2} + \rho \mathbb{E}[W_q].$$

Since $\mathbb{E}[W_q] < \infty$, subtracting $\rho \mathbb{E}[W_q]$ from both sides and dividing by $1 - \rho > 0$ gives the first formula. The second formula is Little’s law applied to the whole system: $L = \lambda(\mathbb{E}[W_q] + \mathbb{E}[S]) = \rho + \lambda \mathbb{E}[W_q]$. $\square$

Corollary 3.6 (Variance increases the wait [12])

Under the assumptions of Theorem 3.5,

$$\mathbb{E}[W_q] = \frac{\rho}{1-\rho} \cdot \frac{1 + \text{CV}^2}{2} \cdot \mathbb{E}[S]. \quad (3.2)$$

Hence, of two work laws with the same λ and the same mean $\mathbb{E}[S]$ (so the same load $\rho < 1$), the one with the larger variance waits strictly longer. In particular:

- (i) M/M/1 ($\text{CV}^2 = 1$): $\mathbb{E}[W_q] = \rho\mathbb{E}[S]/(1 - \rho)$, in agreement with Proposition 2.8.
- (ii) M/D/1 (deterministic service, $\text{CV}^2 = 0$): $\mathbb{E}[W_q] = \rho\mathbb{E}[S]/(2(1 - \rho))$, exactly half of M/M/1 at the same load.

Proof. Substituting $\mathbb{E}[S^2] = \mathbb{E}[S]^2(1 + \text{CV}^2)$ into Eq. (3.1) and using $\lambda\mathbb{E}[S] = \rho$ gives Eq. (3.2). With ρ and $\mathbb{E}[S]$ fixed, the right side is strictly increasing in CV^2 . (i) and (ii) substitute $\text{CV}^2 = 1, 0$. For an exponential law with mean $1/\mu$ this is $\rho/(\mu(1 - \rho))$, the M/M/1 waiting time. $\square$

Remark 3.7

Contrast this with the PS queue of Lecture 2 (Definition 2.9). The mean number in an M/G/1-PS queue is $\rho/(1 - \rho)$, insensitive to the work law (Theorem 2.14), while the mean number in FIFO, Eq. (3.1), is sensitive to the law through $\mathbb{E}[S^2]$. A single long job blocks every customer behind it under FIFO, but not under PS, where all customers share the server.

3.3 Mixtures of hits and misses

The KV cache policy decides whether each turn is a hit or a miss, and hence sets the law of the prefill work.

Definition 3.8 (Hit/miss mixture work)

Let S^{hit} and S^{miss} be square-integrable nonnegative random variables with $S^{\text{miss}} \geq S^{\text{hit}}$ (a.s.). At **hit rate** $h \in [0, 1]$ the work S_h has the law of S^{hit} with probability h and the law of S^{miss} with probability $1 - h$ (the choice is made by an independent coin toss).

Corollary 3.9 (A higher hit rate lowers the PK wait)

Let $m_1(h) = \mathbb{E}[S_h]$, $m_2(h) = \mathbb{E}[S_h^2]$ and $\rho(h) = \lambda m_1(h)$.

- (i) m_1 , m_2 and ρ are affine and nonincreasing in h .
- (ii) If $\rho(h_0) < 1$, then $\rho(h) < 1$ for all $h \in [h_0, 1]$, and $W(h) = \lambda m_2(h)/(2(1 - \rho(h)))$ is nonincreasing on $[h_0, 1]$.

Proof. (i) By the law of total expectation $m_k(h) = h\mathbb{E}[(S^{\text{hit}})^k] + (1-h)\mathbb{E}[(S^{\text{miss}})^k]$, $k = 1, 2$, and its slope in h is $\mathbb{E}[(S^{\text{hit}})^k] - \mathbb{E}[(S^{\text{miss}})^k] \leq 0$ (since $0 \leq S^{\text{hit}} \leq S^{\text{miss}}$, k -th powers preserve the order). (ii) Since ρ is nonincreasing, $\rho(h) \leq \rho(h_0) < 1$. If $h_0 \leq h \leq h'$, then $0 \leq \lambda m_2(h') \leq \lambda m_2(h)$ and $1 - \rho(h') \geq 1 - \rho(h) > 0$, so the numerator decreases and the denominator increases, giving $W(h') \leq W(h)$. $\square$

3.4 The price of a miss

Corollary 3.9 gives only a direction. A scheduler needs a *magnitude*. If we drop the KV state of a particular program i , so that its next turn is a miss, by how much does the delay of the whole prefill stage grow?

Definition 3.10 (Mean number in the prefill stage and policy change)

Model the prefill stage as an M/G/1 FIFO queue with arrival rate λ , work S ($m_1 = \mathbb{E}[S]$, $m_2 = \mathbb{E}[S^2] < \infty$) and load $\rho = \lambda m_1 < 1$. Let $W = \lambda m_2 / (2(1 - \rho))$ be the PK wait and L_P the mean number in the prefill stage (waiting and in service). By Little's law

$$L_P = \lambda(W + m_1) = \rho + \frac{\lambda^2 m_2}{2(1 - \rho)}.$$

For programs (classes) $i = 1, \dots, k$, a **policy change** turns a fraction $q_i \geq 0$ of arrivals ($\sum_i q_i \leq 1$) from hits with work $S_i^{\text{hit}} \geq 0$ into misses with work $S_i^{\text{miss}} = S_i^{\text{hit}} + \Delta S_i$, $\Delta S_i \geq 0$. The moments of the work become

$$m'_1 = m_1 + A, \quad A = \sum_i q_i \Delta S_i, \quad m'_2 = m_2 + B, \quad B = \sum_i q_i ((S_i^{\text{miss}})^2 - (S_i^{\text{hit}})^2),$$

and the new load is $\rho' = \rho + \lambda A$. We write L'_P for the new mean number and $\Delta L_P = L'_P - L_P$ for the change.

Theorem 3.11 (The price of a miss)

In the setting of Definition 3.10, suppose $\rho' < 1$ and define

$$\Phi_i = \Delta S_i + \frac{\lambda((S_i^{\text{miss}})^2 - (S_i^{\text{hit}})^2)}{2(1 - \rho)} + \frac{\lambda W \Delta S_i}{1 - \rho}. \quad (3.3)$$

Then

$$\lambda \sum_i q_i \Phi_i \leq \Delta L_P \leq \frac{1 - \rho}{1 - \rho'} \lambda \sum_i q_i \Phi_i.$$

Proof. Write $u = 1 - \rho > 0$. Then $1 - \rho' = u - \lambda A > 0$, and since $A \geq 0$, $0 < u - \lambda A \leq u$. Applying the PK formula (Theorem 3.5) to the new work,

$$\Delta L_P = \lambda A + \frac{\lambda^2}{2} \left(\frac{m_2 + B}{u - \lambda A} - \frac{m_2}{u} \right) = \lambda A + \frac{\lambda^2}{2} \cdot \frac{u(m_2 + B) - (u - \lambda A)m_2}{u(u - \lambda A)} = \lambda A + \frac{Y}{u(u - \lambda A)},$$

where $Y = \frac{\lambda^2}{2}(uB + \lambda A m_2) \geq 0$. On the other hand, taking the q_i -weighted sum of Eq. (3.3) and using $W = \lambda m_2 / (2u)$,

$$\lambda \sum_i q_i \Phi_i = \lambda A + \frac{\lambda^2 B}{2u} + \frac{\lambda^2 W A}{u} = \lambda A + \frac{\lambda^2}{2u^2} (uB + \lambda A m_2) = \lambda A + \frac{Y}{u^2}.$$

Lower bound. Since $u - \lambda A \leq u$, $Y/(u(u - \lambda A)) \geq Y/u^2$, and hence $\Delta L_P \geq \lambda \sum_i q_i \Phi_i$.

Upper bound. A direct computation gives

$$\frac{u}{u - \lambda A} \left(\lambda A + \frac{Y}{u^2} \right) - \Delta L_P = \lambda A \left(\frac{u}{u - \lambda A} - 1 \right) + \frac{Y}{u(u - \lambda A)} - \frac{Y}{u(u - \lambda A)} = \frac{(\lambda A)^2}{u - \lambda A} \geq 0.$$

Since $u/(u - \lambda A) = (1 - \rho)/(1 - \rho')$, the upper bound holds. $\square$

Reading the three terms. Φ_i is measured in seconds. Misses of program i occur at rate λq_i , so by Little's law $\lambda q_i \Phi_i$ is an increase in the mean number, and Φ_i is “the total waiting time, summed over all turns, that one miss adds”.

- (1) ΔS_i : the miss itself takes longer to serve (its own cost).
- (2) $\lambda((S_i^{\text{miss}})^2 - (S_i^{\text{hit}})^2)/(2(1 - \rho))$: turns that arrive while the miss is being served wait for all of it. This comes from the second moment in the PK formula, and its size is the difference of the *squares* of the work. It is the cost imposed on other customers, the **externality** of the economics of queues; in these notes we call it the **head-of-line term**.
- (3) $\lambda W \Delta S_i / (1 - \rho)$: the extra work raises the load, which stretches every wait by the factor $1/(1 - \rho)$ (the load term).

Since $(S^{\text{miss}})^2 - (S^{\text{hit}})^2 = (S^{\text{miss}} + S^{\text{hit}})\Delta S$,

$$\frac{(2)}{(1)} = \frac{\lambda(S_i^{\text{miss}} + S_i^{\text{hit}})}{2(1 - \rho)}, \quad \frac{(2)}{(3)} = \frac{S_i^{\text{miss}} + S_i^{\text{hit}}}{2W}.$$

The first ratio is of the order of “the number of turns that arrive while one miss is served”. When it is small, the price is close to ΔS_i ; when it is large, the head-of-line term dominates. Once the mean wait W exceeds half the sum of the miss and hit work, the load term exceeds the head-of-line term.

Example 3.12 (At moderate load the head-of-line term dominates)

In seconds, let $\lambda = 1.25$, $m_1 = 0.4$, $m_2 = 0.5$, so $\rho = 0.5$ and $W = 0.625$. Let a program have hit work $S^{\text{hit}} = 0.2$ and miss work $S^{\text{miss}} = 2.0$ ($\Delta S = 1.8$). The three terms are

$$\Delta S = 1.8, \quad \frac{1.25(4 - 0.04)}{2 \cdot 0.5} = 4.95, \quad \frac{1.25 \cdot 0.625 \cdot 1.8}{0.5} = 2.8125,$$

so $\Phi = 9.5625$ seconds. One miss creates more than five times its own extra service in total waiting, and most of it is paid by the turns queued behind it. The two ratios are 2.75 and 1.76. If a fraction $q = 0.01$ of this program's turns turn into misses, then $\rho' = 0.5225$, and the exact change $\Delta L_P \approx 0.1241$ lies in the interval $[0.1195, 0.1252]$ of the theorem.

Corollary 3.13 (Divergence near saturation)

Fix the work law ($m_1 > 0$, $m_2 > 0$) and $\Delta S_i > 0$, and let $\lambda \uparrow 1/m_1$ (that is, $\rho \uparrow 1$). Then $\Phi_i \rightarrow \infty$.

Proof. Put $\rho = 1 - \varepsilon$ with $0 < \varepsilon \leq 1/2$, so $\lambda = (1 - \varepsilon)/m_1 \geq 1/(2m_1)$. The first two terms of Φ_i are nonnegative, and the third is $\lambda W \Delta S_i / \varepsilon = \lambda^2 m_2 \Delta S_i / (2\varepsilon^2) \geq m_2 \Delta S_i / (8m_1^2 \varepsilon^2)$, which diverges as $\varepsilon \rightarrow 0$. $\square$

Corollary 3.14 (Monotonicity in the context length)

Let the prefill cost be $P(n, K) = an + bn(K + n/2)$ ($a, b \geq 0$; the cost of computing n tokens on top of a prefix of length K). A hit costs $S^{\text{hit}} = P(n, K)$ and a miss, which recomputes the whole context, costs $S^{\text{miss}} = P(K + n, 0)$. Then $\Delta S = aK + bK^2/2$, and with λ, ρ, W and the number n of appended tokens fixed, Φ is nondecreasing in $K \geq 0$.

Proof. $P(K + n, 0) - P(n, K) = a(K + n) + \frac{b}{2}(K + n)^2 - an - bnK - \frac{b}{2}n^2 = aK + \frac{b}{2}K^2$. Both ΔS and $S^{\text{hit}} = an + bnK + bn^2/2$ are nonnegative and nondecreasing in K , and so is $S^{\text{miss}} + S^{\text{hit}} = 2S^{\text{hit}} + \Delta S$. The three terms of Eq. (3.3) are nonnegative constants times ΔS , $(S^{\text{miss}} + S^{\text{hit}})\Delta S$ and ΔS , respectively, so all are nondecreasing in K . $\square$

In LLM serving

Chunked prefill splits a long prefill into small chunks and interleaves them with the decode steps, which protects the requests that are decoding. But pending prefills are still processed in arrival order, so a prefill that arrives behind a long miss cannot start until every chunk of that miss is done. As long as the server's order is FIFO, the head-of-line term (2) remains, whatever the chunk size.

In LLM serving

At the same load and with the same append, Corollary 3.14 says that the price of a miss is larger for a program with a longer context. But what the scheduler compares is $p_i\Phi_i/c_i$, divided by the memory c_i and weighted by the resume probability p_i , and that order can flip with the load, because the load changes the weights of the three terms of Φ_i .

Remark 3.15 (Limits of the M/G/1 assumption)

M/G/1 assumes an infinite customer population. In practice a memory pool holds a few dozen live sessions at a time, and a session waiting for prefill cannot create its next turn, so arrivals slow down when the queue is long. The finite population model of Lecture 4 (Proposition 4.6) captures this effect exactly and shows that, for exponential work, the wait of the open formula is at least the finite-population wait at the same utilisation. In that case the W of this lecture is a conservative (upper) estimate. For general work the relation between the two waits is a question for measurement.

3.5 Exercises

Exercise 3.1. Show that the mean residual service time over busy time, $\mathbb{E}[S^2]/(2\mathbb{E}[S])$, is always at least $\mathbb{E}[S]/2$, with equality only when S is deterministic. Also check that it equals $\mathbb{E}[S]$ when S is exponential. *Hint:* $\mathbb{E}[S^2] = \mathbb{E}[S]^2 + \text{Var}[S]$.

Exercise 3.2. With the numbers of Example 3.12 and $q = 0.01$, compute ΔL_P exactly and check $0.1195 \leq \Delta L_P \leq 0.1252$. How does the ratio of the upper to the lower bound change as q grows? *Hint:* substitute $m_1 + A$ and $m_2 + B$ into $L_P = \lambda m_1 + \lambda^2 m_2 / (2(1 - \lambda m_1))$.

Lecture 4

Closed Systems and Memory: Finite Populations, MVA, Campbell's Theorem, Eviction

All models so far were **open systems**. Customers arrive from an infinite outside population, leave after service, and the arrival rate does not change when the queue is long. Agent sessions are different. A session submits a turn, receives the response, runs a tool, and only then submits its next turn. A waiting session cannot create a new turn, so the longer the queue, the fewer the arrivals. This lecture treats this *closed* structure (Section 4.1), computes the mean memory held by sessions (Section 4.2), and solves the combinatorial optimisation problem of what to drop when memory runs short (Section 4.3).

4.1 Closed systems and mean value analysis

Definition 4.1 (Delay station, think time)

A **delay station** or **infinite-server station** is a station with infinitely many servers, so that an arriving customer is served at once without waiting. A customer's sojourn time equals its own service time and does not depend on other customers. In a closed system the time spent at a delay station is called the **think time**, and its mean is written Z .

Definition 4.2 (Closed system, finite-source model)

A **closed system** is a system in which a fixed number N of customers circulate among stations forever, with no entry from or exit to the outside. The **finite-source model** or machine-repairman model $M/M/1//N$ is the closed system made of two stations:

- (a) a delay station with exponential think times of mean Z ,
- (b) a FIFO single server with exponential service times of mean $1/\mu$.

Each customer (session) submits a turn to the server when it finishes thinking, and starts thinking again when its service is done. All think times and service times are mutually independent.

In LLM serving

Sessions are the customers of the closed loop. The think time is the time for tool execution and user response, and the server is the prefill stage. A session waiting for prefill cannot submit its next turn. A KV memory pool holds a few million tokens and real agent contexts reach hundreds of thousands of tokens, so a replica holds about 10 live sessions at a time. At this scale the $N = \infty$ approximation can be badly wrong.

Let $X(t)$ be the number of turns at the server (waiting or in service) at time t . There are $N - X(t)$ thinking sessions, each finishing its think time independently at rate $1/Z$ (Proposition 1.8), so X is a birth–death process on $\{0, 1, \dots, N\}$ with birth rates $\lambda_n = (N - n)/Z$ and death rates $\mu_n = \mu$ ($n \geq 1$). By Theorem 2.6 the stationary law is

$$\pi_N(n) = \frac{1}{G_N} \prod_{k=0}^{n-1} \frac{(N - k)/Z}{\mu} = \frac{1}{G_N} \frac{N!}{(N - n)!} \left(\frac{1}{\mu Z}\right)^n, \quad n = 0, 1, \dots, N, \quad (4.1)$$

with normalising constant G_N . The state space is finite, so no stability condition is needed.

In an open system PASTA (Theorem 1.16) told us that an arriving customer sees the time-average law. In a closed system arrivals are not Poisson, and an arriving customer does not see the time-average law. Instead the following holds.

Theorem 4.3 (Arrival theorem [19, 13])

In M/M/1// N in steady state, the law of the number of turns at the server seen by an arriving turn (excluding itself), that is, the long-run fraction $a_N(n)$ of arrivals that occur in state n , equals the time-stationary law of the same system with $N - 1$ sessions: $a_N(n) = \pi_{N-1}(n)$, $n = 0, \dots, N - 1$.

Proof. In state n arrivals occur at rate $(N - n)/Z$. By the ergodic theorem (Theorem 2.4) and the law of large numbers for jump counts, the number of arrivals in state n during $[0, t]$, divided by t , converges almost surely to $\pi_N(n)(N - n)/Z$. Hence

$$a_N(n) = \frac{(N - n) \pi_N(n)}{\sum_{m=0}^N (N - m) \pi_N(m)}.$$

By (4.1), for $n \leq N - 1$ we have $(N - n) \frac{N!}{(N - n)!} = N \frac{(N - 1)!}{(N - 1 - n)!}$, and for $n = N$, $(N - n) \pi_N(n) = 0$. Therefore $a_N(n) \propto \frac{(N - 1)!}{(N - 1 - n)!} (\mu Z)^{-n} \propto \pi_{N-1}(n)$, and since both are probability distributions they are equal. $\square$

Remark 4.4 (Back to PASTA as $N \rightarrow \infty$)

Fix $\lambda > 0$ and $\mu > \lambda$, and let the think time grow with the population, $Z = N/\lambda$, so that the total arrival rate when all sessions think stays λ . Then $\pi_N(n) \propto \frac{N!}{(N - n)! N^n} \rho^n$ with $\rho = \lambda/\mu$, and $\frac{N!}{(N - n)! N^n} = \prod_{k=0}^{n-1} (1 - k/N)$ lies in $[0, 1]$ and tends to 1. By dominated convergence (with dominating sequence ρ^n) both π_N and π_{N-1} converge to the M/M/1 law $(1 - \rho)\rho^n$. So in the limit the law seen by an arriving turn equals the time-average law: this is PASTA. For a finite N the arrival theorem measures exactly how far the closed system is from that limit.

The arrival theorem lets us solve a closed system by recursion on the number of sessions. With n sessions, let Q_n be the stationary mean number of turns at the server, R_n the mean response time of a turn (wait plus service), and X_n the throughput (turns completed per unit time).

Theorem 4.5 (Mean value analysis [19])

In M/M/1// n , $Q_0 = 0$ and for $n \geq 1$

$$R_n = \frac{1 + Q_{n-1}}{\mu}, \quad X_n = \frac{n}{Z + R_n}, \quad Q_n = X_n R_n.$$

Hence $Q_n = n(1 + Q_{n-1})/(\mu Z + 1 + Q_{n-1})$, the server utilisation is $\rho_n = X_n/\mu = n/(\mu Z + 1 + Q_{n-1})$, and the mean waiting time of a turn (excluding service) is $W_n = Q_{n-1}/\mu$.

Proof. By Theorem 4.3 an arriving turn sees on average Q_{n-1} turns ahead of it. Under FIFO all of them must finish before its service begins. By memorylessness (Proposition 1.8) the remaining time of the turn in service again has mean $1/\mu$, and each waiting turn has mean $1/\mu$, so the mean wait is Q_{n-1}/μ , and the response time is this plus the turn's own service $1/\mu$, which is R_n . One cycle of a session takes $Z + R_n$ on average. Applying Little's law (Theorem 1.19) to the whole system (all n sessions, always n customers) gives $n = X_n(Z + R_n)$, and applying it to the server gives $Q_n = X_n R_n$. The utilisation is $\rho_n = X_n/\mu$ by Corollary 1.20, and substitution gives the remaining formulas. $\square$

Proposition 4.6 (Finite population)

Let $c = \mu Z > 0$ and define Q_n by the recursion of Theorem 4.5.

- (i) Q_n is nondecreasing in n , $0 \leq Q_n \leq n$, and Q_n is nonincreasing in c . Moreover $\rho_N < 1$ for all $N \geq 1$.
- (ii) $W_N \leq \rho_N/(\mu(1 - \rho_N))$. The right side is the open M/M/1 waiting time at the same utilisation (Proposition 2.8).

Proof. Put $g_c(q) = (1 + q)/(c + 1 + q)$, so that $Q_n = n g_c(Q_{n-1})$. On $q \geq 0$, g_c is nondecreasing in q , nonincreasing in c , and $0 < g_c \leq 1$.

(i) $0 \leq Q_n \leq n$ follows by induction from $g_c \leq 1$. $Q_1 = g_c(0) \geq 0 = Q_0$, and if $Q_n \geq Q_{n-1}$ then $Q_{n+1} = (n+1)g_c(Q_n) \geq n g_c(Q_{n-1}) = Q_n$. For $c \leq c'$, assuming $Q_{n-1}(c') \leq Q_{n-1}(c)$, $Q_n(c') = n g_{c'}(Q_{n-1}(c')) \leq n g_{c'}(Q_{n-1}(c)) \leq n g_c(Q_{n-1}(c)) = Q_n(c)$. $\rho_N < 1$ is equivalent to $Q_{N-1} > N - 1 - c$, so it suffices to show $Q_n > n - c$ for all $n \geq 0$ by induction. For $n = 0$ this is $0 > -c$, and a computation gives

$$Q_{n+1} - (n + 1 - c) = \frac{c(Q_n + c - n)}{c + 1 + Q_n} > 0.$$

(ii) Since $\rho_N < 1$, the inequality is equivalent to $Q_{N-1}(1 - \rho_N) \leq \rho_N$, that is, $Q_{N-1} \leq \rho_N(1 + Q_{N-1}) = Q_N$, which is (i). $\square$

Example 4.7

With $N = 2$ and $Z = 2/\mu$ ($c = 2$), $Q_1 = 1/3$, $\rho_2 = 2/(3 + 1/3) = 3/5$ and $W_2 = 1/(3\mu)$. The open formula at the same utilisation gives $\rho_2/(\mu(1 - \rho_2)) = 3/(2\mu)$, which is 4.5 times the actual wait.

Remark 4.8

For exponential work and at the same utilisation, the open formula (Theorem 3.5) and the price of a miss built on it (Theorem 3.11) overestimate the wait (Proposition 4.6(ii)). For general work this comparison has not been proved. Moreover, at most N turns exist, so $0 \leq L_P \leq N$, and no policy change can raise the mean number of turns at the server by more than $N - L_P$. This cap does not depend on the work law.

Remark 4.9

We assumed exponential think times, but the results depend only on their mean Z . The delay station is insensitive like the PS station (Theorem 2.14), and by the BCMP theorem a network of delay stations and PS stations has a product-form stationary law [2]. The service times of a FIFO server do not generalise in this way.

4.2 M/G/∞ and Campbell's theorem

We now count memory. Live sessions do not wait for each other and each keeps its own KV cache, so from the point of view of memory the sessions are customers of a delay station.

Definition 4.10 (M/G/∞)

An M/G/∞ system is a delay station with Poisson arrivals of rate λ (Definition 1.10) and i.i.d. service times S independent of the arrival process.

The following lemma is a standard property of the Poisson process.

Lemma 4.11 (Conditional uniformity [20, §2.3])

For a Poisson process of rate λ , the number of arrivals in an interval $(a, b]$ is $\text{Poisson}(\lambda(b-a))$, and conditional on this number being n , the arrival times (as an unordered set) have the law of n i.i.d. samples from the uniform distribution on $(a, b]$.

Proposition 4.12 (M/G/∞ occupancy [20])

If $\mathbb{E}[S] < \infty$, then in an M/G/∞ system started empty at time $-t$, the number of customers at time 0 is $\text{Poisson}(\lambda \int_0^t \mathbb{P}(S > u) du)$, and as $t \rightarrow \infty$ it converges in distribution to $\text{Poisson}(\lambda \mathbb{E}[S])$.

Proof. Let A be the number of arrivals in $(-t, 0]$. Conditional on $A = n$, the arrival times are i.i.d. uniform (Lemma 4.11) and the service times are independent of them. A customer who arrived at time $-u$ is still present at time 0 only if $S > u$, so each customer remains independently with probability $p_t = \frac{1}{t} \int_0^t \mathbb{P}(S > u) du$. Hence the number X remaining satisfies

$$\mathbb{P}(X = k) = \sum_{n \geq k} e^{-\lambda t} \frac{(\lambda t)^n}{n!} \binom{n}{k} p_t^k (1 - p_t)^{n-k} = e^{-\lambda t p_t} \frac{(\lambda t p_t)^k}{k!}.$$

Finally $\lambda t p_t = \lambda \int_0^t \mathbb{P}(S > u) du \rightarrow \lambda \int_0^\infty \mathbb{P}(S > u) du = \lambda \mathbb{E}[S]$. $\square$

That the limit depends on the law of S only through its mean is the insensitivity of the delay station. When the quantity each customer holds (for example, KV bytes) changes over time, the next theorem gives its mean.

Theorem 4.13 (Campbell's mean formula [10])

Let $\{T_k\}$ be the points of a Poisson process of rate λ on $\mathbb{R}$, and attach to each point an i.i.d. mark $Y_k \sim Y$ (with values in an arbitrary measurable space), independent of the points. For measurable $f \geq 0$,

$$\mathbb{E} \left[\sum_k f(T_k, Y_k) \right] = \lambda \int_{\mathbb{R}} \mathbb{E}[f(t, Y)] dt.$$

Proof. First consider the sum over the points in a bounded interval $I = (a, b]$ only. Conditional on the number of points being n , the pairs (T_k, Y_k) are n i.i.d. copies of (U, Y) with $U \sim \text{Uniform}(I)$ and Y independent (Lemma 4.11). Hence

$$\mathbb{E}\left[\sum_{T_k \in I} f(T_k, Y_k)\right] = \sum_{n \geq 0} \mathbb{P}(A_I = n) n \mathbb{E}[f(U, Y)] = \lambda(b-a) \cdot \frac{1}{b-a} \int_a^b \mathbb{E}[f(t, Y)] dt.$$

(The second equality uses Tonelli's theorem.) For $I_m = (-m, m]$ both sides are nondecreasing in m , and the monotone convergence theorem gives the claim as $m \rightarrow \infty$. $\square$

Corollary 4.14 (Mean resident KV)

Let sessions arrive in a Poisson process of rate Λ , and let session k have an i.i.d. lifetime $T_s^{(k)}$ and KV size $K_k(u) \geq 0$ at age u (the lifetime and the trajectory may be dependent). In steady state (started at time $-\infty$), the mean total resident KV at a given instant is

$$\Lambda \mathbb{E}\left[\int_0^{T_s} K(u) du\right].$$

In particular, if $K(u) \equiv \bar{K}$ is constant for each session, this equals $\Lambda(\mathbb{E}[T_s]\mathbb{E}[\bar{K}] + \text{Cov}(T_s, \bar{K}))$. It exceeds the naive product $\Lambda\mathbb{E}[T_s]\mathbb{E}[\bar{K}]$ if and only if $\text{Cov}(T_s, \bar{K}) > 0$, that is, when long sessions have large contexts.

Proof. Attach the mark $Y_k = (T_s^{(k)}, K_k(\cdot))$ and put $f(t, (T, K)) = K(-t)\mathbf{1}\{0 \leq -t < T\}$. The resident KV at time 0 is $\sum_k f(T_k, Y_k)$. By Theorem 4.13 and Tonelli, its mean is

$$\Lambda \int_{-\infty}^0 \mathbb{E}[K(-t)\mathbf{1}\{-t < T_s\}] dt = \Lambda \mathbb{E}\left[\int_0^{T_s} K(u) du\right].$$

If $K \equiv \bar{K}$, the integral is $T_s \bar{K}$, and $\mathbb{E}[T_s \bar{K}] = \mathbb{E}[T_s]\mathbb{E}[\bar{K}] + \text{Cov}(T_s, \bar{K})$. $\square$

Taking $K \equiv 1$ gives the mean number of sessions $\Lambda\mathbb{E}[T_s]$, in agreement with Little's law.

4.3 Eviction: the covering knapsack problem

When memory runs short, part of the KV of the paused programs must be dropped (**eviction**). Program i occupies $c_i > 0$ bytes, and dropping it has an expected cost $w_i \geq 0$ (for example, the resume probability p_i times the price of a miss, Theorem 3.11).

Definition 4.15 (Covering knapsack problem)

For a finite set I and a target $\Delta C > 0$, the **covering knapsack problem** is

$$\min_{S \subseteq I} w(S) := \sum_{i \in S} w_i \quad \text{s.t.} \quad c(S) := \sum_{i \in S} c_i \geq \Delta C.$$

Allowing $x \in [0, 1]^I$ in place of S and minimising $\sum w_i x_i$ subject to $\sum c_i x_i \geq \Delta C$ gives the **fractional version**. Engines drop blocks rather than programs, so the real problem is close to the fractional version. We call $v_i = w_i/c_i$ the **price per byte**.

Theorem 4.16 (Threshold rule [1])

- (i) For every $\theta \geq 0$, the set $T_\theta = \{i : v_i \leq \theta\}$ costs no more than any S with $c(S) \geq c(T_\theta)$.
- (ii) In the fractional version, the solution that sets $x_i = 1$ in increasing order of v_i and fills the last item partially to meet the target exactly is optimal (provided $c(I) \geq \Delta C$).

Proof. (i) On $S \setminus T_\theta$ we have $w_i > \theta c_i$, and on $T_\theta \setminus S$ we have $w_i \leq \theta c_i$, so

$$w(S) - w(T_\theta) = \sum_{S \setminus T_\theta} w_i - \sum_{T_\theta \setminus S} w_i \geq \theta(c(S \setminus T_\theta) - c(T_\theta \setminus S)) = \theta(c(S) - c(T_\theta)) \geq 0.$$

- (ii) Let x^* be the greedy solution and θ the price per byte of the last (partial) item. Then $x_i^* = 1$ if $v_i < \theta$, $x_i^* = 0$ if $v_i > \theta$, and $\sum c_i x_i^* = \Delta C$. For any feasible x ,

$$\sum_i w_i(x_i - x_i^*) = \sum_i (w_i - \theta c_i)(x_i - x_i^*) + \theta \sum_i c_i(x_i - x_i^*).$$

Each term of the first sum is nonnegative (if $v_i < \theta$ both factors are ≤ 0 , if $v_i > \theta$ both are ≥ 0 , and if $v_i = \theta$ the term is 0). The second sum is $\theta(\sum c_i x_i - \Delta C) \geq 0$. $\square$

In the integer version, price-per-byte order can be arbitrarily bad. With target $\Delta C = 2$ and three items $(c, w) = (1, 1), (L, L), (2, 3)$, price-per-byte order (ties broken in favour of the earlier item) takes the first two items at cost $1 + L$, while the third item alone costs 3. The ratio $(1 + L)/3$ diverges as $L \rightarrow \infty$. The standard repair is the *guarded greedy* algorithm, which guesses the most expensive evicted item and fills the rest in price-per-byte order; it is known to be within a factor 2 of the optimum for all w_i [3].

Real systems evict by a key fixed in advance rather than by a price: shortest context first (shortest-first, SF), least recently used (LRU), time-to-live expiry (TTL), and so on. Such keys do not look at the resume probability p_i .

Proposition 4.17 (Price-blind keys)

Consider the covering knapsack problem with $w_i = p_i \Phi_i$.

- (i) Consider a rule that chooses one of two programs from any information except their resume probabilities. For every R there are resume probabilities $p \in (0, 1]$ under which either program alone meets the target and the program the rule chooses costs more than R times the other. Hence no such rule has a constant approximation ratio.
- (ii) If every program resumes ($p_i = 1$) and the price is only the quadratic term of the recompute work, so that $w_i = c_i^2$, then SF equals price-per-byte order, costs at most 2 times any feasible solution, and the constant 2 cannot be improved.

Proof. (i) Take two programs of the same size $c > 0$ that agree in everything the rule observes, and a target $\Delta C = c$. The rule's input does not depend on the resume probabilities, so it drops a fixed one, e ; call the other o . Set $\Phi_e = \Phi_o = \Phi > 0$, $p_e = 1$ and $p_o = 1/(|R| + 2)$. Then $w_e = \Phi > R\Phi/(|R| + 2) = R w_o$. The optimum is at most w_o , so the rule's ratio exceeds R .

(ii) Since $w_i/c_i = c_i$, price-per-byte order is SF. SF takes programs from the shortest until the target is met, so it returns a feasible solution whenever the pool holds the target. Suppose SF takes $t_1 \leq \dots \leq t_m$ in sorted order, and let $\ell = t_m$ and $\sigma = t_1 + \dots + t_{m-1} < \Delta C$ (it did not stop earlier). Let S be any feasible solution.

Step 1: $\ell^2 \leq w(S)$. Every program with $c_i < \ell$ is among the first $m - 1$, and these free only $\sigma < \Delta C$. So S contains a program with $c_i \geq \ell$.

Step 2. Put $f(x) = x(\ell - x)$ for $x < \ell$ and $f(x) = 0$ for $x \geq \ell$. Then $f \geq 0$ and $x^2 \geq \ell x - f(x)$ for all $x \geq 0$. Summing over S and using $c(S) \geq \Delta C$, and noting that the terms with $f(c_i) > 0$

occur only among $t_1, \dots, t_{m-1}$,

$$w(S) \geq \ell \Delta C - \sum_{j < m} t_j(\ell - t_j) = \ell(\Delta C - \sigma) + \sum_{j < m} t_j^2.$$

Step 3. Since $\Delta C - \sigma > 0$, $w(\text{SF}) = \sum_{j < m} t_j^2 + \ell^2 \leq w(S) - \ell(\Delta C - \sigma) + \ell^2 \leq w(S) + \ell^2 \leq 2w(S)$.

Optimality of the constant 2: with sizes $K, K+1$ and $\Delta C = K+1$, SF takes K and then $K+1$ at cost $K^2 + (K+1)^2$, while $\{K+1\}$ costs $(K+1)^2$. The ratio tends to 2 as $K \rightarrow \infty$. $\square$

Example 4.18

Even in the situation of (ii), SF is not optimal. With contexts 4, 5, 6 and target 6, SF drops $\{4, 5\}$ at cost $16 + 25 = 41$, while $\{6\}$ is feasible at cost 36.

Part (i) says that when resume probabilities differ between programs, no fixed key can stand in for the price. How large such differences are in practice is a question about the workload and must be measured.

4.4 Exercises

Exercise 4.1. For $N = 2$ and $c = \mu Z$, compute $\sum_n n \pi_2(n)$ directly from (4.1) and check that it equals Q_2 of Theorem 4.5. *Hint:* $G_2 = 1 + 2/c + 2/c^2$.

Exercise 4.2. Show that the stationary mean number of customers in $M/G/\infty$ is $\lambda \mathbb{E}[S]$ without using Proposition 4.12. *Hint:* at a delay station $W = \mathbb{E}[S]$, so use Little's law.

Lecture 5

Putting It Together: The Theory of the Paper, Step by Step

The first four lectures built tools. This lecture assembles them into the argument of the paper. We go in the order the paper does: the model (Section 5.1), where the variance of prefill work comes from (Section 5.2), the decision problem and its single price (Section 5.3), and the scheduler whose every step compares that price with the price of memory (Section 5.4). We close with an honest list of what is exact and what is approximate (Section 5.5). For each step we name the result of Lectures 1–4 that carries it.

5.1 The model in one page

Definition 5.1 (Replica and its three demands)

The system has J **replicas**; each has a memory pool of M bytes. A turn i has context K_i tokens (already in the session), appends n_i new tokens and decodes o_i output tokens. It makes three demands on the replica that serves it.

- (a) *Prefill compute*. Prefilling n tokens on top of a resident prefix of K tokens costs

$$P(n, K) = a n + b n (K + n/2) \quad (5.1)$$

seconds, with $a, b \geq 0$: a linear term for the dense layers and an attention term over the prefix. A **hit** costs $P(n_i, K_i)$; a **miss** recomputes the whole context, $P(K_i + n_i, 0)$.

- (b) *Decode bandwidth*. $D_i = o_i(\beta K_i + \omega/m)$ seconds, where m is the batch size, β the time to read one token's KV and ω the time to read the weights once.

- (c) *Memory*. $c_i = \kappa K_i$ bytes while the state is resident, with κ the KV bytes per token.

The hit/miss distinction lives entirely in demand (a): D_i and c_i are the same for a hit and a miss. By Corollary 3.14 the extra work of a miss is $\Delta S_i = a K_i + b K_i^2/2$, quadratic in the context.

Two stations and a pool. Figure 1.1 in Section 1.2 shows the structure; here we add the numbers and name the lecture that backs each modelling choice.

1. *Decode is a PS station* with capacity $\varphi(m)$ (Definition 2.9). Continuous batching advances every turn in the batch at each step. By Theorem 2.14 the batch-size law is $\pi(m) \propto \rho_D^m / \prod_{k \leq m} \varphi(k)$ with $\rho_D = \lambda \mathbb{E}[D]$, whatever the law of D .
2. *Prefill is a FIFO queue* whose server is available the fraction of time the decode batch leaves. Chunked prefill interleaves chunks with decode steps but serves pending prefills

in arrival order, so the M/G/1 FIFO analysis of Lecture 3 applies, with the PK wait $\mathbb{E}[W_q] = \lambda \mathbb{E}[S^2] / (2(1 - \rho))$ (Theorem 3.5).

3. *Sessions form a closed loop* (prefill $\rightarrow$ decode $\rightarrow$ tool), and new sessions arrive as a Poisson process of rate Λ . The tool call is a delay station (Definition 4.1).
4. *The pool* of M bytes holds both the running batch and the paused sessions' KV. Once it is full, the KV policy decides what to drop, and so which future turns hit.

The objective. The paper minimises $L = L_P + L_D$, the mean number of turns in the prefill stage plus the mean number in the decode batch. By Little's law (Theorem 1.19), L equals λ times the mean time a turn spends in the two stages, so minimising L is minimising mean delay.

In LLM serving

The two stations respond to the cache in opposite ways. The decode stage sees only the mean demand (insensitivity), and a miss does not change the decode demand at all (Corollary 2.17). The prefill stage sees the second moment of its work (PK), and the cache sets that second moment. This asymmetry is the whole reason the paper prices misses at the prefill queue.

5.2 Where the variance of prefill work comes from

The PK formula makes $\mathbb{E}[S^2]$, equivalently CV^2 , a first-class quantity (Corollary 3.6). The paper does not assume that agentic workloads are intrinsically “high variance”; it asks where the variance comes from. The tool is the law of total variance.

Lemma 5.2 (Variance split of the hit/miss mixture)

Let S_h be the mixture of Definition 3.8: S^{hit} with probability h and S^{miss} with probability $1 - h$, with means $\mu_{\text{hit}}, \mu_{\text{miss}}$ and variances $\sigma_{\text{hit}}^2, \sigma_{\text{miss}}^2$. Then

$$\text{Var}[S_h] = \underbrace{h \sigma_{\text{hit}}^2 + (1 - h) \sigma_{\text{miss}}^2}_{\text{spread of the appends}} + \underbrace{h(1 - h) (\mu_{\text{miss}} - \mu_{\text{hit}})^2}_{\text{hit/miss mixture}}.$$

Proof. Let I be the indicator of a hit. By the law of total variance, $\text{Var}[S_h] = \mathbb{E}[\text{Var}(S_h | I)] + \text{Var}(\mathbb{E}[S_h | I])$. The first term is $h \sigma_{\text{hit}}^2 + (1 - h) \sigma_{\text{miss}}^2$. The conditional mean $\mathbb{E}[S_h | I]$ takes the value μ_{hit} with probability h and μ_{miss} with probability $1 - h$, a two-point variable whose variance is $h(1 - h)(\mu_{\text{miss}} - \mu_{\text{hit}})^2$. $\square$

The first term is not the scheduler's to change: it comes from the spread of tool outputs (a line or a whole file). The second term is the KV policy's lever. It vanishes at $h \in \{0, 1\}$ and grows with the squared gap $(\mu_{\text{miss}} - \mu_{\text{hit}})^2$, which by (5.1) grows like K^4 once attention dominates. The paper measures both terms on production traces rather than assuming either one dominates. The lesson for the theory is that what a scheduler must price is not a ratio of CV^2 but the rise of $\lambda \mathbb{E}[S^2] / (2(1 - \rho))$ that one more miss causes. That is exactly Φ_i .

5.3 The decision problem and one price

At a decision epoch a replica holds a set of paused programs. Program i holds c_i bytes, resumes with probability p_i after an expected further τ_i seconds, and its next turn needs S_i^{hit} if its state is still resident and $S_i^{\text{hit}} + \Delta S_i$ if not. The controller decides for each state whether to keep it, and the memory constraint couples all these decisions.

Step 1: price a miss. Theorem 3.11 gives the rise in L_P when a fraction of turns turns from hits into misses:

$$\Phi_i = \underbrace{\Delta S_i}_{\text{own work}} + \underbrace{\frac{\lambda((S_i^{\text{miss}})^2 - (S_i^{\text{hit}})^2)}{2(1-\rho)}}_{\text{head-of-line}} + \underbrace{\frac{\lambda W \Delta S_i}{1-\rho}}_{\text{load}}.$$

A single eviction is a transient event, while Theorem 3.11 compares two stationary regimes. The paper therefore reads one eviction as a small change of policy, a fraction q_i of arrivals turning from hits into misses. This is an approximation, and the paper tests it empirically.

Step 2: price a drop. Dropping program i 's state causes a miss only if the program resumes, so the expected price of the drop, its price per byte and its price per byte-second held are

$$w_i = p_i \Phi_i, \quad v_i = \frac{w_i}{c_i}, \quad u_i = \frac{w_i}{c_i \tau_i}. \quad (5.2)$$

Step 3: price memory. Keeping program i occupies c_i bytes for about τ_i seconds, that is $c_i \tau_i$ byte-seconds. Let $x_i \in \{0, 1\}$ indicate “keep” and let B be the byte-seconds available over the horizon. The keep-or-drop problem is

$$\text{OPT} = \min_{x \in \{0,1\}^I} \sum_i (1 - x_i) w_i \quad \text{s.t.} \quad \sum_i c_i \tau_i x_i \leq B. \quad (5.3)$$

Relaxing the constraint with a multiplier $\theta \geq 0$ gives the Lagrangian $\mathcal{L}(x, \theta) = \sum_i (1 - x_i) w_i + \theta (\sum_i c_i \tau_i x_i - B)$. We call θ the **shadow price** of a byte-second.

Proposition 5.3 (One price for memory)

- (i) For every $\theta \geq 0$, $x_i^\theta = \mathbf{1}\{u_i > \theta\}$ minimises $\mathcal{L}(\cdot, \theta)$, and $g(\theta) := \mathcal{L}(x^\theta, \theta) \leq \text{OPT}$.
- (ii) If θ is such that x^θ uses the budget exactly, $\sum_i c_i \tau_i x_i^\theta = B$, then x^θ is optimal for (5.3).

Proof. (i) $\mathcal{L}(x, \theta) = \sum_i w_i - \theta B + \sum_i x_i (\theta c_i \tau_i - w_i)$ is separable in x . Each term is minimised by $x_i = 1$ when $\theta c_i \tau_i < w_i$, that is $u_i > \theta$, and by $x_i = 0$ otherwise. For any feasible x the bracket $\sum_i c_i \tau_i x_i - B$ is ≤ 0 , so $\sum_i (1 - x_i) w_i \geq \mathcal{L}(x, \theta) \geq g(\theta)$. Minimising over feasible x gives $\text{OPT} \geq g(\theta)$ (weak duality).

(ii) x^θ is feasible, and its constraint term vanishes, so its objective equals $g(\theta) \leq \text{OPT}$. Hence it is optimal. $\square$

Part (ii) is the threshold rule of Theorem 4.16 in byte-second form: keep what is worth more per byte-second than memory, drop the rest. When no θ uses the budget exactly (items are indivisible), the fractional relaxation is still solved by the threshold rule with one item split, and engines that evict blocks rather than whole programs are close to that case.

Step 4: how long to keep. Proposition 5.3 decides at one instant. Because the Lagrangian is separable, once θ is fixed each program can also be treated on its own over time: keep its state until some time T and drop it then if the program has not resumed. Let the program resume with probability $p \in (0, 1]$, and, given that it resumes, after a time with a continuous distribution function G and density g on $[0, \infty)$. Holding the state costs θc per second; dropping it costs Φ if the program resumes later. The expected cost of the rule “hold until T ” is

$$C(T) = \theta c \int_0^T (1 - pG(t)) dt + \Phi p (1 - G(T)), \quad (5.4)$$

since the state is held at time t exactly when the program has not resumed by t , and a miss occurs exactly when it resumes after T . The quantity that decides is the **resume hazard**

$$\eta(t) = \frac{p g(t)}{1 - pG(t)},$$

the rate at which a program that has not resumed by t resumes at t .

Proposition 5.4 (Optimal holding time)

Let $r = \theta c / \Phi$ with $\Phi > 0$, and suppose g is continuous.

- (i) $C'(T) = (1 - pG(T))(\theta c - \Phi \eta(T))$ for every $T \geq 0$. Holding a little longer lowers the cost exactly when $\Phi \eta(T) > \theta c$.
- (ii) If η is nonincreasing, let $T^* = \inf\{T \geq 0 : \eta(T) \leq r\}$. If $T^* < \infty$ it minimises C over $[0, \infty)$; if $T^* = \infty$, C is nonincreasing and the state should be held until the program resumes. In particular $T^* = 0$, drop at once, if and only if $\eta(0) \leq r$.
- (iii) If G is exponential with mean τ , then η is nonincreasing and $\eta(0) = p/\tau$, so the state should be kept at all if and only if

$$u = \frac{p\Phi}{c\tau} > \theta,$$

the price per byte-second of (5.2). If $p = 1$ and $u > \theta$, the state is held until the program resumes. If $p < 1$ and $u > \theta$, the optimal holding time is

$$T^* = \tau \ln \left[\frac{p}{1-p} \left(\frac{\Phi}{\theta c \tau} - 1 \right) \right].$$

Proof. (i) The integrand in (5.4) is continuous, so by the fundamental theorem of calculus the first term has derivative $\theta c(1 - pG(T))$, and the second has derivative $-\Phi p g(T)$. Factoring out $1 - pG(T)$ gives the formula when $pG(T) < 1$; when $pG(T) = 1$ (possible only if $p = 1$) both sides vanish, because $g(T) = 0$ wherever G has reached 1 and g is continuous. The factor $1 - pG(T)$ is nonnegative, so the sign of $C'(T)$ is the sign of $\theta c - \Phi \eta(T)$.

(ii) For $T < T^*$, $\eta(T) > r$, so $C'(T) \leq 0$ by (i). For $T > T^*$, $\eta(T) \leq r$ because η is nonincreasing, so $C'(T) \geq 0$. Hence C is nonincreasing on $[0, T^*]$ and nondecreasing on $[T^*, \infty)$, and T^* is a minimiser. If $T^* = \infty$, then $C' \leq 0$ everywhere. The last claim is the definition of T^* at $T = 0$, using that η is nonincreasing.

(iii) With $x = e^{-T/\tau}$, $g(T) = x/\tau$ and $1 - pG(T) = 1 - p + px$, so

$$\eta(T) = \frac{p}{\tau} \cdot \frac{x}{1 - p + px}.$$

The map $x \mapsto x/(1 - p + px)$ is nondecreasing on $(0, 1]$, and x decreases in T , so η is nonincreasing; it is the constant $1/\tau$ when $p = 1$. At $T = 0$, $\eta(0) = p/\tau$, and $\eta(0) > r$ is $p\Phi/(c\tau) > \theta$. If $p = 1$, $\eta \equiv 1/\tau > r$, so $T^* = \infty$ by (ii). If $p < 1$, $\eta(T) \rightarrow 0$ as $T \rightarrow \infty$, so T^* is finite and solves $\eta(T^*) = r$ by continuity: $px/\tau = r(1 - p + px)$, that is $x = r(1 - p)/(p(1/\tau - r))$, and $T^* = -\tau \ln x = \tau \ln \left[\frac{p}{1-p} \left(\frac{1}{r\tau} - 1 \right) \right]$ with $1/(r\tau) = \Phi/(\theta c \tau)$. $\square$

So the instantaneous test $u_i > \theta$ of the scheduler is exactly the optimal decision to keep a state for exponential resume times, and part (iii) adds how long to keep it. For example, with $p = 0.5$, $\tau = 10$ s, $\Phi = 20$ s, $c = 1$ and $\theta = 0.3$, $u = 1 > \theta$, the optimal holding time is $T^* \approx 17.3$ s, and it lowers the expected cost from $C(0) = p\Phi = 10$ to $C(T^*) \approx 5.6$. When the gap distribution has a decreasing hazard, as heavy-tailed gaps typically do, part (ii) gives a program-specific time-to-live.

Remark 5.5 (Unknown resume times and the oracle)

If nothing is known about G , holding the state is the classical *ski rental* problem: pay rent θc per second or buy once at price Φ [6]. The rule “hold for $\Phi/(\theta c)$ seconds” costs at most twice the cost in hindsight for any resume time: if the program resumes at $\tau \leq \Phi/(\theta c)$ it pays $\theta c\tau$, which is optimal; otherwise it pays 2Φ against a hindsight optimum of Φ . The hindsight optimum, which knows whether and when each program resumes, is called the **oracle**; it keeps a state exactly when the program resumes and $\theta c\tau < \Phi$. It cannot be run online, but it is the benchmark against which a policy’s cost is reported as cost/OPT . The factor two needs the program to resume: against a program that never does, the oracle pays nothing, and no rule that holds the state for a positive time has a bounded ratio.

Proposition 5.4 holds θ and Φ fixed. In the real system both move with the policy: what is dropped changes the load, and the load changes every Φ_i and the value of θ that fills the pool. The joint optimum is a Markov decision problem over the whole state, with no closed form; Proposition 5.4 is its solution at a fixed point of (θ, Φ) .

5.4 The scheduler, step by step

Every decision now compares a program’s u_i with the replica’s θ .

Eviction. When ΔC bytes must be freed now, the problem is the covering knapsack of Definition 4.15 with $w_i = p_i\Phi_i$. Take blocks in increasing u_i until ΔC is freed; θ is the u_i of the last block taken, and $\theta = 0$ while the pool has room. Deployed rules evict by a fixed key: shortest context first, least recently used, longest idle, or an expired time-to-live. Proposition 4.17 says what such a key can and cannot do: a key that ignores p_i has no constant approximation ratio (part (i)), and shortest-first is within a factor two only when every program resumes and the price is the recompute work alone (part (ii)).

Example 5.6 (The load reorders programs)

Take $a = 0.2$ ms/token and $b = a/K_c$ with $K_c = 50,000$ tokens, and an append of $n = 1000$ tokens. Program A has $K = 200$ k tokens and $p = 0.3$; program B has $K = 20$ k tokens and $p = 0.9$. Then $\Delta S_A = 120$ s and $\Delta S_B = 4.8$ s. At an idle queue ($\lambda \rightarrow 0$) the price is the own work, and the price per thousand tokens $p\Phi/K$ is 0.18 s for A and 0.216 s for B: evict A first. At $\lambda = 0.25$ /s with $\mathbb{E}[S] = 2$ s and $\mathbb{E}[S^2] = 20$ s² ($\rho = 0.5$, $W = 5$ s) the head-of-line term dominates; $\Phi_A \approx 4080$ s and $\Phi_B \approx 23.2$ s, and the prices per thousand tokens become 6.12 s and 1.05 s: evict B first. No fixed key reproduces both orders, which is why the scheduler prices at decision time.

Keep, offload or drop. With a slower tier (host memory or a shared KV store) each paused state has three options: keep it at cost $\theta c_i\tau_i$, drop it at expected cost w_i , or offload it at the **priced fetch**, which is Φ_i with the transfer time in place of ΔS_i (plus the tier’s own wait once it saturates). The scheduler takes the least of the three. Two facts follow from this being a minimum.

Proposition 5.7 (Offloading as an option)

Let each state i have a finite set of actions $\mathcal{A}_i$ with costs $\gamma_i(a)$, and let the total cost be separable. Then

- (i) enlarging each $\mathcal{A}_i$ never raises the optimal cost; in particular, making offloading available never hurts;
- (ii) a policy that always offloads can cost more than one that never offloads.

Proof. (i) A minimum over a larger set is at most the minimum over a subset. (ii) One state with keep cost 1, drop cost 1 and priced fetch 2 suffices: always offloading pays 2, never offloading pays 1. $\square$

So when an always-offloading stack collapses, the collapse is a property of the policy, not of the offloading mechanism.

Placement. A resuming turn can run where its state is (replica 1) or elsewhere (replica j). The router charges each replica its wait W_j and service S_j , the cost R_j of moving or recomputing the state, and the expected future cost F_j of the program's remaining turns, and picks the minimum. The price of a miss supplies R_j : moving turns the next turn into a miss served at replica j , so R_j is Φ_i at replica j 's load. *Session affinity*, the rule that never moves a program, sets $R_j = \infty$ for $j \neq 1$.

Proposition 5.8 (When affinity loses)

Let the affinity replica be an M/M/1 queue with service rate μ and utilisation $\rho < 1$, and let the alternative be an idle replica with the same rate, at extra cost $x = R + F \geq 0$. Staying costs more than moving if and only if

$$\rho > \rho^* := \frac{\mu x}{1 + \mu x},$$

and ρ^* is nondecreasing in x .

Proof. By Proposition 2.8 the mean response time at the affinity replica is $1/(\mu(1 - \rho))$, and moving costs $1/\mu + x$. The inequality $1/(\mu(1 - \rho)) > 1/\mu + x$ is $1/(1 - \rho) > 1 + \mu x$, that is $1 - \rho < 1/(1 + \mu x)$, that is $\rho > \mu x/(1 + \mu x)$. The map $x \mapsto \mu x/(1 + \mu x) = 1 - 1/(1 + \mu x)$ is nondecreasing. $\square$

An idle alternative gives the earliest inversion, so ρ^* is a lower bound on the load at which affinity loses to any real alternative. A shared KV store lowers R and hence ρ^* : affinity and a store are alternatives, not complements.

Admission. The pool holds both the batch and the paused states, and the two compete unstably: a miss keeps its turn longer in the prefill queue and in the batch, which holds more bytes, which evicts more paused state, which causes more misses. Admission keeps the replica out of this regime by capping the number of live sessions. The cap comes from Corollary 4.14: with sessions of lifetime T_s holding $K(t)$ tokens at age t , the mean resident KV is $\kappa \Lambda \mathbb{E}[\int_0^{T_s} K(t) dt]$. Writing $\bar{k} = \kappa \mathbb{E}[\int_0^{T_s} K(t) dt] / \mathbb{E}[T_s]$ for the time-averaged footprint of a live session and using Little's law for the mean number of live sessions $\Lambda \mathbb{E}[T_s]$, the mean resident KV is (live sessions) $\times \bar{k}$. The admission step caps live sessions near $M/\bar{k}$ and holds the rest in an entry queue. Corollary 4.14 also warns that the naive footprint $\kappa \mathbb{E}[\bar{K}]$ underestimates $\bar{k}$ whenever long contexts belong to

long sessions. The cap has a price of its own, the wait at the entry queue, and a session is worth admitting while the memory it will hold, priced at θ per byte-second, costs less than that wait.

The scheduler in one table. Each deployed rule replaces one price by a constant:

Rule	What it fixes
shortest-first eviction	$w_i \propto c_i^2$ (every program resumes, own work only)
LRU, time-to-live	p_i constant
always offload	priced fetch = 0
strict session affinity	$R_j = \infty$
admit every session	$\theta = 0$

5.5 What is exact and what is approximate

A mathematician should know which statements are theorems about the model and which are modelling choices.

- *Exact, given the model:* the price bracket (Theorem 3.11), the decode price and its monotonicity (Theorem 2.12, Proposition 2.16), the finite-population inequalities (Proposition 4.6), the price-blind bounds (Proposition 4.17), and Propositions 5.3, 5.7 and 5.8.
- *Approximate:* the prefill stage is not M/G/1. Session feedback makes its arrivals non-Poisson, and the server’s budget fluctuates with the decode batch. BCMP product form covers the PS and delay stations but not the FIFO prefill stage. For exponential work, Proposition 4.6(ii) shows the open formula overstates the wait at equal utilisation. For general work this comparison is an empirical question.
- *Approximate:* one eviction is a transient event priced by a stationary comparison (Step 1 above).
- *Not in the queueing formula:* a turn whose KV does not fit waits for memory blocks before its prefill starts. That wait is set by the pool, not by the queue, and adds to the PK wait. The capacity function φ depends on the footprint law and the admission rule and must be calibrated, not derived.

What the testbed showed about the last point. On the paper’s testbed with contexts of about 50k tokens, the wait for the first token was a wait for memory blocks: the prefill server was busy a small fraction of the time, and the PK wait was one to two orders of magnitude below the observed one. Three engine rules made the pool a queue. A waiting turn is admitted only when its whole prompt fits; admission is in arrival order, so a turn that does not fit blocks everyone behind it (head-of-line blocking, now on memory); and every admission takes the least recently used cached blocks. What was observed: the turns that missed had the same think times as the hits, and nearly all of them had arrived while their rank already had a queue. What a post hoc model of the replica suggests, but the measurements do not show directly: the admissions ahead of them evicted their prefixes while they waited. Either way the misses were selected by congestion, so on that testbed the ratio of miss to hit waiting times measures congestion, not the price of a miss. In the short-context runs the extra misses caused by forced ones followed long think gaps and a higher rate of re-inserted contexts, which points to the pool (re-insertions shortening how long a cached prefix survives) rather than to the wait. In queueing terms, the hit rate is not a policy input but depends on the wait, and the wait on the hit rate; the next step of this research treats this feedback.

The paper tests the approximate links in two ways: in simulation on replayed production sessions, and on a real testbed. The criterion is *decision faithfulness*: the model must rank policies the way the real system does, even where its latencies are off.

5.6 Exercises

Exercise 5.1. In Lemma 5.2 let $\sigma_{\text{hit}} = \sigma_{\text{miss}} = \sigma$. Find the hit rate h that maximises the mixture term, and show that the mixture term exceeds the append term exactly when $h(1 - h)(\mu_{\text{miss}} - \mu_{\text{hit}})^2 > \sigma^2$. *Hint:* $h(1 - h)$ is maximised at $h = 1/2$.

Exercise 5.2. With $\mu = 1$ turn/s, compute ρ^* of Proposition 5.8 for $x = 0.1, 1$ and 10 s, and interpret: how loaded must the affinity replica be before moving a program with a ten-second recompute pays off? *Hint:* $\rho^* = x/(1 + x)$ when $\mu = 1$.

Bibliography

- [1] S. Asmussen, *Applied Probability and Queues*, 2nd ed., Springer, 2003.
- [2] F. Baskett, K. M. Chandy, R. R. Muntz, F. G. Palacios, Open, closed, and mixed networks of queues with different classes of customers, *Journal of the ACM* 22(2), 1975.
- [3] J. Csirik, J. B. G. Frenk, M. Labbé, S. Zhang, Heuristics for the 0-1 min-knapsack problem, *Acta Cybernetica* 10, 1991.
- [4] G. B. Dantzig, Discrete-variable extremum problems, *Operations Research* 5(2), 1957.
- [5] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*, Cambridge University Press, 2013.
- [6] A. R. Karlin, M. S. Manasse, L. Rudolph, D. D. Sleator, Competitive snoopy caching, *Algorithmica* 3, 1988.
- [7] F. P. Kelly, *Reversibility and Stochastic Networks*, Wiley, 1979.
- [8] A. Khinchine, Mathematical theory of a stationary queue, *Matematicheskii Sbornik*, 1932.
- [9] A. Y. Khinchine, *Mathematical Methods in the Theory of Queueing*, Griffin, London, 1960.
- [10] J. F. C. Kingman, *Poisson Processes*, Oxford University Press, 1993.
- [11] L. Kleinrock, *Queueing Systems, Volume 2: Computer Applications*, Wiley, 1976.
- [12] L. Kleinrock, *Queueing Systems, Volume 1: Theory*, Wiley, 1975.
- [13] E. D. Lazowska, J. Zahorjan, G. S. Graham, K. C. Sevcik, *Quantitative System Performance*, Prentice-Hall, 1984.
- [14] J. D. C. Little, A proof for the queueing formula $L = \lambda W$, *Operations Research* 9(3), 1961.
- [15] R. M. Loynes, The stability of a queue with non-independent inter-arrival and service times, *Proc. Cambridge Philosophical Society* 58(3), 1962.
- [16] P. Naor, The regulation of queue size by levying tolls, *Econometrica* 37(1), 1969.
- [17] J. R. Norris, *Markov Chains*, Cambridge University Press, 1997.
- [18] F. Pollaczek, Über eine Aufgabe der Wahrscheinlichkeitstheorie, *Mathematische Zeitschrift*, 1930.
- [19] M. Reiser, S. S. Lavenberg, Mean-value analysis of closed multichain queueing networks, *Journal of the ACM* 27(2), 1980.
- [20] S. M. Ross, *Stochastic Processes*, 2nd ed., Wiley, 1996.
- [21] A. Agrawal et al., Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve, 2024.
- [22] S. Stidham, A last word on $L = \lambda W$, *Operations Research* 22(2), 1974.
- [23] R. W. Wolff, Poisson arrivals see time averages, *Operations Research* 30(2), 1982.